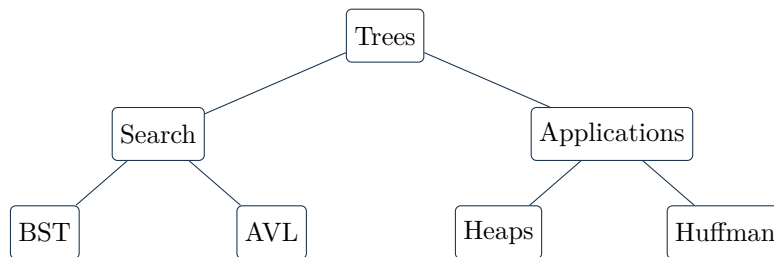


Non-Linear Data Structures II

Visual explanations, numbered algorithms,
selective C, and practice with solutions



Scope from the supplied topic list

Tree introduction and binary-tree terminology; memory representation; binary-tree traversals; construction from traversals; binary search tree insertion and deletion; AVL rotations and insertion; tree applications; heapify; heap sort; Huffman trees; analysis of algorithms. **How these notes are written**

Concepts come first. Each main operation has numbered Start-to-End steps. Selected short C examples connect those steps to implementation. Diagrams, traces, and plain-English explanations show what changes and why.

How to use this guide

Read Sections 1–4 before studying the search trees. Draw each example yourself, cover the next step, and predict what happens. For heaps and Huffman trees, keep both the picture and the sequence of values visible. Finish with the practice problems before reading the answers.

The concept sections require only comparing numbers, adding small values, and reading a sequence. C examples are optional reading after the diagram and algorithm. They are compact functions, with no menu programs or lengthy input-handling code.

How to read an algorithm. Follow the numbered steps in order unless a step tells you to repeat or skip ahead. Start and End mark the boundaries. “Apply this algorithm to a subtree” means complete a smaller call and then return to the next step of the paused call. Ending a subtree call does not end the whole traversal.

What is new in this revision. Explicit algorithms cover traversal, reconstruction, BST updates, rotations, AVL insertion, heap operations, heap sort, and Huffman coding. Numbered tree pictures make visit order visible. Additional traces show how traversal returns upward and how insertion attaches a new leaf. Short C examples cover the central implementation patterns.

Conventions used throughout

- A root is at **depth 0**. Depth counts edges from the root.
- Height also counts edges. A leaf has height 0; an empty tree has height -1 when used in height formulas.
- A tree of height h has $h + 1$ occupied levels if it is nonempty.
- Search-tree examples use **distinct keys** unless duplicates are discussed explicitly.
- Inorder means left subtree, node, right subtree. Left and right are different positions.
- Hand-worked heap examples use **one-based array positions**. C heap snippets use **zero-based indices**, as C arrays do. Both formulas are given; the convention is identified beside the snippets.
- AVL balance factor means left height minus right height.
- n usually means the number of nodes or items. In Huffman analysis, k is the number of distinct symbols and N is the number of input symbols.

A note about the diagrams. A line connects a parent to a child. A missing branch means that child does not exist. Small lettered blocks in rotation diagrams stand for entire subtrees; they can contain many nodes or be empty.

Contents

1	Trees: the basic picture	2
1.1	What is a data structure?	2
1.2	What makes a structure a tree?	2
1.3	Terminology without mystery	2
1.4	Small facts that solve many questions	3
1.5	Why trees are useful	3

2	Binary trees and their representation in memory	4
2.1	What makes a tree binary?	4
2.2	Important shapes	4
2.3	Counting nodes, levels, and leaves	5
2.4	Linked representation: store connections explicitly	5
2.5	A small C foundation: one node and its children	6
2.6	Array representation: let positions imply the connections	7
2.7	Choosing between the two	7
3	Binary-tree traversals: visiting every node	8
3.1	What traversal means	8
3.2	The three depth-first orders	8
3.3	One example, all four traversals	8
3.4	How to trace without losing your place	9
3.5	Graphical visit order: the same tree, four different journeys	9
3.6	Precise traversal algorithms	9
3.7	Inorder slowed down: movement is not the same as a visit	10
3.8	Short C examples: only the visit line changes position	11
3.9	The stack idea behind depth-first traversal	11
3.10	The queue idea behind level order	11
3.11	When each traversal helps	12
3.12	Traversal analysis and checks	13
4	Constructing a binary tree from traversals	14
4.1	The information each traversal gives	14
4.2	Construction from preorder and inorder	14
4.3	Construction from postorder and inorder	15
4.4	Why preorder and postorder are not generally enough	16
4.5	What about inorder and level order?	16
4.6	Spotting inconsistent inputs	16
4.7	Time and space analysis	16
5	Binary search trees: search, insertion, and deletion	18
5.1	The rule that makes searching possible	18
5.2	Searching as repeated elimination	18
5.3	Insertion: search until the new position is found	19
5.4	Insertion slowed down: finding a place versus changing a link	19
5.5	Short C insertion: returning the root of each subtree	20

5.6	Insertion order changes the shape	21
5.7	Minimum, maximum, predecessor, and successor	21
5.8	Deletion case 1: the node is a leaf	22
5.9	Deletion case 2: the node has one child	22
5.10	Deletion case 3: the node has two children	22
5.11	Short C deletion: the three cases in one function	24
5.12	Why successor replacement preserves the order	24
5.13	BST operation costs	24
6	AVL trees: keeping a search tree short	26
6.1	The problem AVL trees solve	26
6.2	Balance must hold everywhere	26
6.3	What a rotation actually does	26
6.4	C rotations: a small number of changed references	27
6.5	The four insertion cases	28
6.6	A complete AVL insertion routine in words	29
6.7	Short C AVL insertion: BST choices plus local repair	30
6.8	Why AVL height is logarithmic	31
6.9	AVL deletion: a short boundary note	32
7	Binary heaps and heapify	33
7.1	The two separate rules of a binary heap	33
7.2	Why the complete shape matters	33
7.3	Heapify: repairing one local violation	33
7.4	Sift down, step by step	34
7.5	Building a heap from an unordered array	34
7.6	A graphical link between the array and its tree	35
7.7	Short C heap repair and construction	36
7.8	Why bottom-up build-heap is linear	36
7.9	Insertion into a heap: sift up	36
7.10	Removing the root	37
7.11	Heap operation summary and applications	38
8	Heap sort: sorting by repeatedly removing the largest	39
8.1	The main idea	39
8.2	A complete trace	39
8.3	Short C heap sort: shrinking the active region	40
8.4	Why it works	40

8.5	Time and memory	40
8.6	Stability and direction of sorting	40
9	Huffman trees: shorter codes for frequent symbols	42
9.1	The problem being solved	42
9.2	The prefix-free rule	42
9.3	A code tree	42
9.4	Huffman's construction rule	42
9.5	Calculating the total cost	44
9.6	A powerful cross-check: sum the merge weights	45
9.7	Encoding and decoding	45
9.8	Why combining the smallest weights makes sense	46
9.9	Ties, edge cases, and non-uniqueness	46
9.10	Huffman analysis	47
10	Analysis of algorithms in simple English	48
10.1	What are we measuring?	48
10.2	Growth rates you should recognize	48
10.3	What a logarithm means here	48
10.4	Big O, Omega, and Theta	49
10.5	Best, worst, and average cases	49
10.6	Why lower-order terms are dropped	49
10.7	Useful sums and recurrences	49
10.8	Space: stored data versus temporary help	50
10.9	How to write a good analysis answer	50
11	Connections, applications, and revision sheets	51
11.1	Which structure solves which problem?	51
11.2	Applications explained through their main operation	51
11.3	Operation-cost reference	51
11.4	Compact formula sheet	52
11.5	Frequent exam mistakes and their corrections	52
11.6	A suggested learning sequence	53
11.7	How to connect an algorithm, a picture, and a C snippet	53
12	Practice problems: draw, reason, calculate	55
12.1	Foundations and counting: Problems 1–6	55
12.2	Memory representation: Problems 7–10	55

12.3 Traversals: Problems 11–16	56
12.4 Construction from traversals: Problems 17–22	56
12.5 Binary search trees: Problems 23–29	56
12.6 AVL trees: Problems 30–36	57
12.7 Heaps and heapify: Problems 37–44	57
12.8 Heap sort: Problems 45–48	58
12.9 Huffman coding: Problems 49–54	58
12.10 Analysis and mixed reasoning: Problems 55–60	59
13 Answers and worked solutions	60
13.1 Solutions 1–10: foundations and memory	60
13.2 Solutions 11–16: traversals	60
13.3 Solutions 17–22: reconstruction	61
13.4 Solutions 23–29: BST operations	62
13.5 Solutions 30–36: AVL trees	63
13.6 Solutions 37–44: heaps	64
13.7 Solutions 45–48: heap sort	65
13.8 Solutions 49–54: Huffman coding	65
13.9 Solutions 55–60: analysis and synthesis	66
13.10 Additional implementation checks, with explanations	67
Final self-test checklist	68

1 Trees: the basic picture

1.1 What is a data structure?

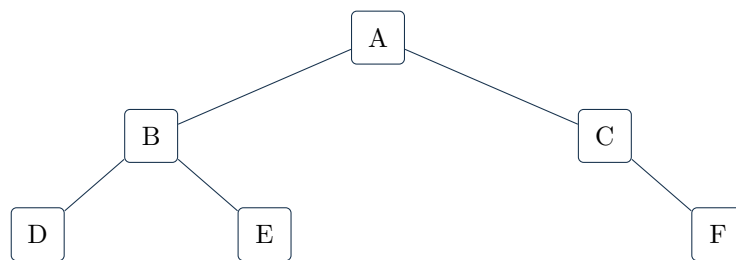
A data structure is a way to arrange information so that useful tasks become easier. A row of books supports reading from first to last. A dictionary arranges words so that finding a word is easier. Different arrangements make different operations convenient.

A **linear** structure puts items in a sequence. A **non-linear** structure allows branching relationships. In a tree, one item can lead to several smaller groups. A family tree, a folder hierarchy, and the rounds of a knockout tournament all suggest this branching shape. A real family history can be more complicated than a strict data-structure tree, so the analogy is only a starting point.

1.2 What makes a structure a tree?

A nonempty tree is a connected arrangement of nodes with no cycles. A **node** is an item, and an **edge** is a connection between two nodes. Connected means every node can be reached from every other node by following connections. No cycles means you cannot follow distinct edges around a loop and return to your starting point.

For the rooted trees in this unit, we choose one node as the **root** and draw it at the top. Every other node has exactly one parent. There is exactly one simple path from the root to any node. “Simple” means the path does not repeat a node.



Read this picture as follows: A is the root. A has children B and C. B has children D and E. C has only a right child, F. D, E, and F have no children.

1.3 Terminology without mystery

Term	Meaning and example
Root	The starting node at the top. A is the root.
Parent	The node immediately above another node. B is the parent of E.
Child	A node directly below a parent. E is a child of B.
Siblings	Nodes with the same parent. D and E are siblings.
Leaf	A node with no children. D, E, and F are leaves.
Internal node	A node with at least one child. A, B, and C are internal nodes.
Ancestor	A node above another on the root-to-node path. A and B are ancestors of E. We exclude the node itself here.
Descendant	A node below another. D and E are descendants of B. We exclude B itself.

Term	Meaning and example
Subtree	A node together with all its descendants and their connecting edges. The subtree rooted at B contains B, D, and E.
Degree of a node	Number of children in a rooted tree. B has degree 2; C has degree 1. This differs from degree in an undirected graph, which counts all incident edges.
Degree of a tree	Maximum number of children of any node. This tree has degree 2.
Path	A sequence of connected nodes. A–B–E is a path.
Path length	Number of edges along the path. A–B–E has length 2.
Depth	Number of edges from the root. A has depth 0; E has depth 2.
Height of a node	Largest number of edges from that node down to a leaf. B has height 1.
Height of a tree	Height of its root. The example has height 2.
Forest	A collection of separate trees. Removing A leaves the B subtree and the C subtree as a forest.

Key idea. Depth looks upward toward the root. Height looks downward toward the deepest leaf. They are not interchangeable.

1.4 Small facts that solve many questions

A nonempty tree with n nodes has exactly $n - 1$ edges. Why? Every node except the root has one parent connection. There are $n - 1$ non-root nodes, so there are $n - 1$ such connections.

Removing any edge splits a tree into two parts. Adding an edge between two nodes already in the same tree creates a cycle. These facts follow from there being exactly one simple path between any two nodes.

An empty tree has no nodes and no edges. A one-node tree has one root, which is also a leaf, and height 0. Many mistakes come from forgetting these small cases.

1.5 Why trees are useful

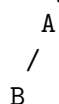
Trees help when information is hierarchical, when choices repeatedly divide into smaller choices, or when a structure should keep its depth small. They can represent folders, arithmetic expressions, searchable records, priorities, and compact symbol codes. The best tree depends on the operation: a search tree helps find a particular key; a heap helps repeatedly take the smallest or largest item.

2 Binary trees and their representation in memory

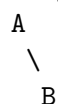
2.1 What makes a tree binary?

A **binary tree** gives every node at most two child positions: left and right. A node can have no children, just a left child, just a right child, or both. A left-only child and a right-only child are different arrangements.

Left-only child:



Right-only child:



The word binary says how many child positions are allowed. It does **not** say that numbers are sorted, that the tree is balanced, or that every parent has two children.

2.2 Important shapes

Full, strict, or proper binary tree: every node has either zero or two children. Nobody has exactly one child.

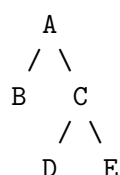
Complete binary tree: all levels except possibly the last are full, and the last level is occupied from left to right with no gaps. This is the shape used by a binary heap.

Perfect binary tree: every internal node has two children and all leaves are at the same depth. Every level is full.

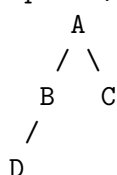
Skewed binary tree: every internal node has just one child, producing a chain. An all-left chain or all-right chain is a special case.

Balanced tree: an informal description meaning the height is kept small. A precise balancing rule must be stated. AVL trees use a specific local height rule, explained later.

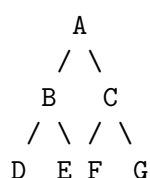
Full, but not complete:



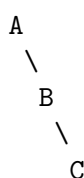
Complete, but not full:



Perfect:



Right-skewed:



Every perfect binary tree is both full and complete. A full tree need not be complete. A complete tree need not be full. A binary search tree need not have any of these special shapes.

Common mistake. “Full” and “complete” sound similar in everyday English, but they are different definitions. Some textbooks vary the names; check the definition, not just the label.

2.3 Counting nodes, levels, and leaves

At depth d , there can be at most 2^d nodes: 1 at depth 0, 2 at depth 1, 4 at depth 2, and so on. Each node can create at most two positions at the next depth.

For a nonempty binary tree of height h :

$$\underbrace{h+1}_{\text{a chain}} \leq n \leq \underbrace{1+2+4+\dots+2^h}_{\text{every level full}} = 2^{h+1} - 1.$$

For a fixed $n \geq 1$, the smallest possible height is

$$\lceil \log_2(n+1) \rceil - 1 = \lfloor \log_2 n \rfloor,$$

and the largest possible height is $n-1$. A complete binary tree achieves the smallest height. Here, floor means round down and ceiling means round up.

A perfect binary tree of height h has 2^h leaves and $2^h - 1$ internal nodes.

There is also a useful rule for *every nonempty binary tree*. Let n_0, n_1, n_2 count nodes with zero, one, and two children. Counting edges in two ways gives

$$n-1 = n_1 + 2n_2, \quad n = n_0 + n_1 + n_2.$$

Subtracting gives $n_0 = n_2 + 1$. In words, the number of leaves is one more than the number of nodes with two children. For a full binary tree, all internal nodes have two children, so leaves = internal nodes + 1 and total nodes = twice the internal nodes + 1.

Worked example: quick counting

A perfect tree of height 3 has $1+2+4+8=15$ nodes and 8 leaves. A full tree with 9 internal nodes has 10 leaves and 19 total nodes. A complete tree with 10 nodes has height $\lfloor \log_2 10 \rfloor = 3$, so it has four occupied levels.

2.4 Linked representation: store connections explicitly

Think of each node as a small record with three parts:

```
+-----+-----+-----+
| link to left | stored value | link to right |
+-----+-----+-----+
```

A **link** or pointer is a reference telling us where another node can be found. Nodes need not sit next to one another in memory. A special empty marker, often called **null**, means there is no child. A separate root reference tells us where the tree starts. If the root reference is null, the tree is empty.

Record	Left link	Value	Right link
Record 1	Record 2	A	Record 3
Record 2	empty	B	empty
Record 3	Record 4	C	empty
Record 4	empty	D	empty

This represents A with left child B and right child C; C has left child D. “Record 1” is a label for a location, not the stored value.

For n nodes, there are $2n$ child-link slots. Exactly $n - 1$ hold real edges, so $n + 1$ child-link slots are empty. This count does not include the separate root reference.

Advantages: natural for irregular shapes; no storage is reserved for all the missing positions of a deep tree; changing a few connections can insert, remove, or rotate nodes.

Costs: each node needs link storage; following references may jump between distant memory locations; a node cannot be reached just by calculating a position from its value. Linked storage still uses $\Theta(n)$ space, but with overhead per node.

Some designs add a parent link, height, count of duplicate keys, or subtree size. These are additional fields, not requirements of an ordinary binary tree.

2.5 A small C foundation: one node and its children

This is the shared node type used by the later tree snippets. A `Node *` is a reference to a node; `NULL` means there is no node. The arrow in `p->left` means “the left-child reference stored in the node p.” It does not move or copy a whole subtree.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int key;
    struct Node *left, *right;
    int height;           /* Used by the AVL snippets. */
} Node;

Node *new_node(int key) {
    Node *p = malloc(sizeof *p);
    if (p == NULL) {
        fputs("Cannot allocate a tree node.\n", stderr);
        exit(EXIT_FAILURE);
    }
    p->key = key;
    p->left = p->right = NULL;
    p->height = 0;
    return p;
}
```

Read this in English. Reserve space for one node. If that is impossible, stop with a message. Otherwise store the key, make both child references empty, record leaf height 0, and return the new node’s reference. `malloc` reserves the space; `free`, used later, releases it.

The `height` field is included so the same node type can support AVL examples later. The ordinary BST snippets do not maintain that field or AVL balance. Build and update an AVL tree using the AVL operations, which do maintain it. All snippets here store integer keys only; associated records would need to move with their keys.

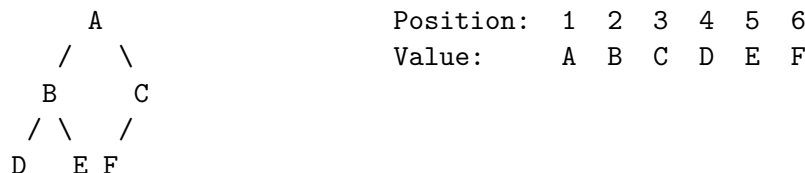
2.6 Array representation: let positions imply the connections

An **array** is a row of numbered storage positions. Store the root first, then reserve positions level by level. For **one-based positions**:

$$\text{left child of } i = 2i, \quad \text{right child of } i = 2i + 1, \quad \text{parent of } i = \lfloor i/2 \rfloor \quad (i > 1).$$

For **zero-based positions**:

$$\text{left} = 2i + 1, \quad \text{right} = 2i + 2, \quad \text{parent} = \lfloor (i - 1)/2 \rfloor \quad (i > 0).$$



For a complete tree with n nodes, a calculated child position exists exactly when it is at most n in the one-based convention. In a sparse tree stored with explicit gaps, being within the allocated range is insufficient: that position may be empty.

Worked example: why gaps can waste memory

In a right-only chain A–B–C–D, positions are 1, 3, 7, 15. Four values require positions through 15 if we use this direct positional scheme. For a right chain of n nodes, the last position is $2^n - 1$. A linked representation needs only n records. Other compact tree encodings exist, but they are different representations.

2.7 Choosing between the two

Question	Linked nodes	Positional array
Irregular or skewed shape	Usually suitable	May waste many slots
Complete tree	Works	Especially suitable
Finding parent	Needs a parent link or search/path	Simple position formula
Changing local shape	Update links	May require repositioning values
Storage overhead	Child references per node	No child references required
Typical use here	BST and AVL trees	Binary heaps

3 Binary-tree traversals: visiting every node

3.1 What traversal means

A traversal visits all nodes in a defined order. “Visit” means perform the required action at a node: write its label, count it, inspect it, or process its stored value. Moving past a node while travelling through the picture does not necessarily mean visiting it at that moment.

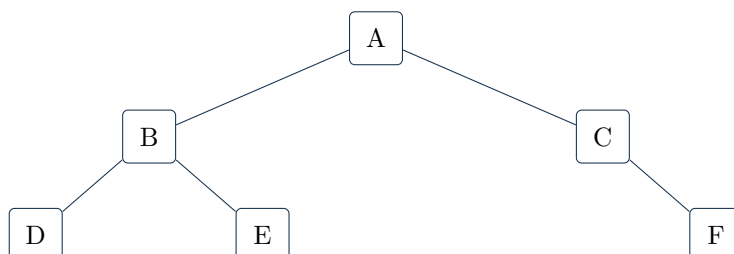
Three common traversals work by finishing smaller subtrees. They are **depth-first**: go down a branch before completing nearby branches. The fourth, level order, works across the tree one level at a time.

3.2 The three depth-first orders

Traversal	Order	Memory aid
Preorder	Node, left, right	Root comes before its subtrees
Inorder	Left, node, right	Root comes between its subtrees
Postorder	Left, right, node	Root comes after its subtrees

These instructions apply again inside every subtree. An empty subtree contributes nothing. A leaf contributes just its own label, whichever order you use.

3.3 One example, all four traversals



Preorder: visit A first. Finish B’s subtree as B, D, E. Then finish C’s subtree as C, F. The answer is

A, B, D, E, C, F .

Inorder: finish B’s subtree as D, B, E; visit A; finish C’s subtree as C, F. The answer is

D, B, E, A, C, F .

Postorder: finish B’s subtree as D, E, B; finish C’s subtree as F, C; visit A last. The answer is

D, E, B, F, C, A .

Level order: read depth 0, then depth 1, then depth 2, left to right within each level. The answer is

A, B, C, D, E, F .

3.4 How to trace without losing your place

For a depth-first traversal, draw a small box around the left subtree and another around the right subtree. Decide where the root belongs relative to those two boxes. Solve the contents of each box using the same rule. This replaces one large problem with two smaller problems.

For example, inorder of the whole tree is

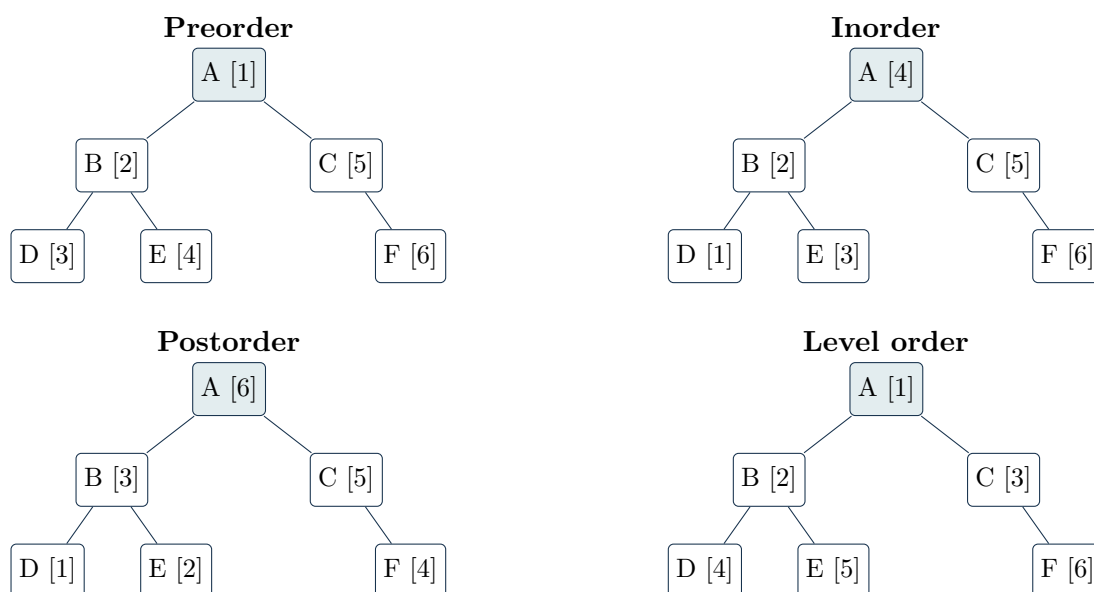
(inorder of B subtree), A , (inorder of C subtree).

The B part becomes $(D), B, (E)$ and the C part becomes (empty), $C, (F)$.

The idea of applying the same procedure to a smaller version of the problem is called **recursion**. You can understand and use it without writing code. The smallest stopping case is an empty subtree or a leaf.

3.5 Graphical visit order: the same tree, four different journeys

In each node below, the label identifies the node and the bracketed number gives its **visit number**. The edges still show parent-child relationships; the numbers show processing order. The tree's shape never changes during a traversal.



3.6 Precise traversal algorithms

Input: a reference to the root of the current tree or subtree. **Output:** each stored value visited once in the required order. A visit may mean printing the value, as in the C examples, or adding it to a result.

Algorithm: Preorder traversal

- Step 1. Start.** Receive the current subtree's root.
- Step 2.** If it is empty, go to Step 6.
- Step 3.** Visit the current root.
- Step 4.** Apply this preorder algorithm to its left subtree. Wait for that call to finish.

Step 5. Apply this preorder algorithm to its right subtree. Wait for that call to finish.

Step 6. End this call. Return to the caller; if this was the original call, traversal is complete.

Algorithm: Inorder traversal

Step 1. Start. Receive the current subtree's root.

Step 2. If it is empty, go to Step 6.

Step 3. Apply this inorder algorithm to its left subtree. Wait for that call to finish.

Step 4. Visit the current root.

Step 5. Apply this inorder algorithm to its right subtree. Wait for that call to finish.

Step 6. End this call. Return to the caller; if this was the original call, traversal is complete.

Algorithm: Postorder traversal

Step 1. Start. Receive the current subtree's root.

Step 2. If it is empty, go to Step 6.

Step 3. Apply this postorder algorithm to its left subtree. Wait for that call to finish.

Step 4. Apply this postorder algorithm to its right subtree. Wait for that call to finish.

Step 5. Visit the current root.

Step 6. End this call. Return to the caller; if this was the original call, traversal is complete.

3.7 Inorder slowed down: movement is not the same as a visit

Here is the full reasoning for the six-node example. An empty-child call immediately finishes; it produces no output.

Stage	What happens and why	Output so far
1	Enter A. Inorder requires its left subtree first, so pause A before its visit.	empty
2	Enter B. Pause B and enter its left subtree D.	empty
3	D's left is empty. Visit D. Its right is empty, so D finishes.	D
4	Return to B. Its left subtree is finished, so now visit B.	D, B
5	Enter B's right subtree E. Its left is empty; visit E; its right is empty. E finishes.	D, B, E
6	B has finished both children and its own visit. Return to A. A's whole left subtree is now complete.	D, B, E
7	Visit A, then enter A's right subtree C.	D, B, E, A
8	C has no left child. Visit C, then enter its right subtree F.	D, B, E, A, C
9	F has no children. Visit F and finish its call.	D, B, E, A, C, F
10	C finishes, then A finishes. There is no paused caller left.	complete

Key idea. Returning to B does not automatically mean visiting B again. The paused algorithm resumes at its next unfinished step. In preorder B was visited before descending; in inorder it is visited between its child calls; in postorder it is visited after both.

3.8 Short C examples: only the visit line changes position

These three functions use the earlier `Node` definition. Read `return`; as “finish this call and resume whoever called it.” The empty-tree check prevents following a child reference from a nonexistent node.

```
void preorder(const Node *root) {
    if (root == NULL) return;
    printf("%d ", root->key);
    preorder(root->left);
    preorder(root->right);
}

void inorder(const Node *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->key);
    inorder(root->right);
}

void postorder(const Node *root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->key);
}
```

Connection to the steps. The empty check is Step 2. The two subtree calls and the print are Steps 3–5. Reaching the closing brace finishes Step 6. For a leaf, both child calls return immediately, so its value is printed exactly once.

3.9 The stack idea behind depth-first traversal

Imagine a pile of reminder cards. Before entering a subtree, leave a card saying where to return and what remains to be done. When that subtree finishes, use the most recent reminder first. This last-in, first-out arrangement is called a **stack**.

For inorder, repeatedly move left while keeping reminders of the nodes you passed. When you can go no farther left, visit the most recently remembered node, then work through its right subtree. For preorder, visit a node before descending. For postorder, delay the visit until both subtrees are complete.

The exact number of reminders depends on the implementation and tree shape, but ordinary recursive traversal uses $O(h + 1)$ extra space. In a balanced tree that is $O(\log n)$; in a chain it can be $O(n)$.

3.10 The queue idea behind level order

A **queue** is a waiting line: the item that joined first is handled first. Put the root in the waiting line. Repeatedly remove the first node, visit it, and put its existing children at the back, left child

before right child.

Node just visited	Waiting line afterward	Output so far
None	A	empty
A	B, C	A
B	C, D, E	A, B
C	D, E, F	A, B, C
D	E, F	A, B, C, D
E	F	A, B, C, D, E
F	empty	A, B, C, D, E, F

The queue can contain parts of two neighbouring levels at once. If w is the maximum number of nodes on any one level, its size is $O(w)$, and in the worst case $O(n)$.

Algorithm: Level-order traversal

Input: the root. **Output:** values level by level, from left to right.

Step 1. Start. Receive the root reference.

Step 2. If the root is empty, go to Step 10.

Step 3. Create an empty waiting line and place the root at its back.

Step 4. If the waiting line is empty, go to Step 10.

Step 5. Remove the node at the front and call it the current node.

Step 6. Visit the current node.

Step 7. If its left child exists, place that child at the back of the waiting line.

Step 8. If its right child exists, place that child at the back of the waiting line.

Step 9. Repeat from Step 4.

Step 10. End. Every reachable node has been visited once.

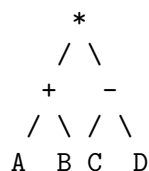
Why these steps preserve level order. A node's children join behind nodes already waiting at its level. Thus all waiting parents are handled before their newly added children. The earlier queue table shows the algorithm's complete trace. A full queue implementation in C would add storage-management detail without changing this tree procedure, so the compact C examples focus on the three depth-first orders.

3.11 When each traversal helps

- **Preorder:** record a parent before its descendants, or represent an expression with each operator before its inputs. Saving a general binary tree uniquely also requires shape information, such as empty-child markers.
- **Inorder:** obtain keys in sorted order from a BST. Inorder of an arbitrary binary tree is not necessarily sorted.

- **Postorder:** finish children before handling their parent, useful for expression evaluation, subtree calculations, and safely removing an entire tree.
- **Level order:** display nodes by depth and explore all nearby nodes before deeper ones.

Worked example: an expression tree



This represents $(A + B) \times (C - D)$. Preorder is $*, +, A, B, -, C, D$. Postorder is $A, B, +, C, D, -, *$. Inorder gives $A, +, B, *, C, -, D$, but parentheses must be restored to preserve the intended grouping. With $A = 2, B = 3, C = 9, D = 5$, the left subtree gives 5, the right subtree gives 4, and the root gives 20.

3.12 Traversal analysis and checks

Every complete traversal takes $\Theta(n)$ time if visiting a node takes constant time: each of the n nodes is visited exactly once. It is not $O(\log n)$ just because the tree is balanced. Traversal reads everything; search may follow only one path.

Storing the final sequence needs $\Theta(n)$ output space. This is separate from the stack or queue needed during traversal. A useful answer states whether it includes output storage.

Common mistake. In preorder, the root of *every subtree* comes first, not just the overall root. Apply the full rule at every node.

Self-checks: each node appears once; preorder starts with the root; postorder ends with the root; inorder places every left-subtree node before the root and every right-subtree node after it.

4 Constructing a binary tree from traversals

4.1 The information each traversal gives

Preorder immediately identifies the root: it is first. Postorder identifies the root: it is last. Inorder identifies which nodes lie in the left and right subtrees once the root is known. This is why combining inorder with either preorder or postorder is powerful.

In this section, node labels are distinct. If labels repeat, a root may occur in more than one inorder position, making the split ambiguous unless extra identity information is supplied.

4.2 Construction from preorder and inorder

Algorithm: Construct from preorder and inorder

Input: matching preorder and inorder segments with distinct labels. **Output:** the subtree they describe, or an inconsistency report.

Step 1. Start. Receive the two current segments.

Step 2. If both are empty, return an empty subtree and end this call. If their lengths or label sets differ, report inconsistent input and end this call.

Step 3. Take the first preorder label as the new root label.

Step 4. Find that label in the current inorder segment. If it is absent, report inconsistent input and end this call.

Step 5. Split inorder before and after the root. These are the left and right groups. Let L be the number of labels in the left group.

Step 6. In preorder, the next L labels after the root are the left group; all remaining labels are the right group.

Step 7. Create the root. Apply this same algorithm to the left preorder and left inorder groups; attach the result as its left subtree.

Step 8. Apply the algorithm to the two right groups; attach that result as its right subtree. If a child call reports inconsistency, report the whole input inconsistent rather than accepting a partial tree.

Step 9. End this call. Return the constructed subtree root.

The label-set checks make the validity conditions explicit. For an already validated input, the efficient construction method uses segment boundaries and a position lookup; repeatedly constructing sets or copying groups is not part of its linear-time implementation.

Worked example: preorder and inorder, with every split shown

Preorder: A, B, D, E, C, F

Inorder: D, B, E, A, C, F

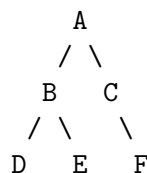
Step 1: choose A. It is first in preorder. Inorder splits as

$$(D, B, E) \quad A \quad (C, F).$$

The left subtree has three nodes, so preorder after A splits into (B, D, E) and (C, F) .

Step 2: build the left subtree. Its preorder starts with B. Its inorder is $(D), B, (E)$, so B has left child D and right child E.

Step 3: build the right subtree. Its preorder starts with C. Its inorder is $(\text{empty}), C, (F)$, so C has no left child and has right child F.



Step 4: verify. Reading preorder gives A, B, D, E, C, F. Reading inorder gives D, B, E, A, C, F. Both match the supplied sequences.

4.3 Construction from postorder and inorder

The idea is the same, except the root is the last postorder label. Split inorder around it. The left-subtree count determines how many postorder items belong to the left subtree; the right-subtree items follow them, just before the root.

Algorithm: Construct from postorder and inorder

Input: matching postorder and inorder segments with distinct labels. **Output:** their subtree, or an inconsistency report.

- Step 1. Start.** Receive the two current segments.
- Step 2.** If both are empty, return an empty subtree and end this call. If their lengths or label sets differ, report inconsistent input and end this call.
- Step 3.** Take the last postorder label as the new root label.
- Step 4.** Locate it in the current inorder segment. If it is absent, report inconsistent input and end this call.
- Step 5.** Split inorder into the group before the root and the group after it. Count the left group's labels as L .
- Step 6.** In postorder, the first L labels form the left group. The remaining labels before the final root form the right group.
- Step 7.** Create the root. Apply this algorithm to the two left groups and attach the returned left subtree.
- Step 8.** Apply it to the two right groups and attach the returned right subtree. Propagate any inconsistency instead of accepting a partial result.
- Step 9. End this call.** Return the completed subtree root.

Worked example: postorder and inorder

Postorder: D, E, B, F, C, A

Inorder: D, B, E, A, C, F .

A is last in postorder, so it is the root. Inorder gives left group D, B, E and right group C, F. The postorder groups are therefore D, E, B and F, C. B is the left group's last label, so it becomes the left root. C is the right group's last label, so it becomes the right root. The final tree is the same as the previous example.

If consuming one postorder sequence *backward*, handle root, then right subtree, then left subtree. If working with separate left and right segments on paper, either segment may be solved first.

4.4 Why preorder and postorder are not generally enough

Tree 1:



Tree 2:



Both trees have preorder A, B and postorder B, A. The sequences do not reveal which side contains the single child. Therefore these traversals do not uniquely determine a general binary tree.

With **distinct labels and the additional promise that the tree is full**, preorder and postorder do determine a unique binary tree. A non-leaf root must have two children, so the single-child ambiguity disappears. The next preorder label identifies the left-subtree root, and locating it in postorder reveals the size of that subtree. This is an extra condition, not the general rule.

Inorder alone does not give the shape. For example, inorder A, B can describe root A with right child B or root B with left child A. Preorder alone and postorder alone also do not identify a general binary tree without extra information.

4.5 What about inorder and level order?

For distinct labels, a valid inorder and level-order pair also determines a unique binary tree. The first level-order label is the root. Split inorder around it, then retain the left-subtree labels from level order in their existing order to obtain that subtree's level order. Do the same for the right subtree. Repeat. Straightforward repeated filtering can take $O(n^2)$ time; the ordinary preorder/inorder and postorder/inorder methods are usually the main syllabus focus.

4.6 Spotting inconsistent inputs

Before drawing, check that the two traversals have the same length and contain the same labels. During construction, every chosen root must occur inside the corresponding inorder segment, and each subtree segment must contain the matching set of labels. Equal lengths and equal overall label sets are necessary but not sufficient.

For example, preorder A, B, C and inorder C, A, B cannot describe the same binary tree. Root A would need left subtree {C}; however, the next preorder item is B, which cannot be in that left subtree.

4.7 Time and space analysis

If you scan inorder from the beginning of the current segment to find each root, a skewed case can require work proportional to

$$n + (n - 1) + (n - 2) + \cdots + 1 = \Theta(n^2).$$

If a position lookup is prepared once, root positions can be found in constant time per node under the usual constant-time lookup assumption. Using segment boundaries rather than repeatedly copying sequences then gives $O(n)$ total construction time.

The new tree uses $\Theta(n)$ space. The position lookup uses $O(n)$ extra space, and recursive reminders use $O(h + 1)$. If a question excludes the result tree, state the remaining auxiliary space separately. Efficient lookup alone is not enough if the procedure still repeatedly copies large segments.

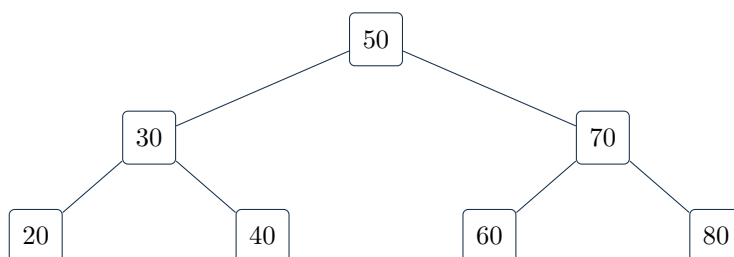
5 Binary search trees: search, insertion, and deletion

5.1 The rule that makes searching possible

A **key** is the value used to compare records. In a binary search tree, or BST, every node obeys this rule:

- Every key anywhere in its left subtree is smaller than its key.
- Every key anywhere in its right subtree is larger than its key.
- Both subtrees obey the same rule.

This is the distinct-key convention. For duplicate keys, choose an explicit policy, such as storing an occurrence count with one node. Using counts is especially convenient in balanced trees.



Key idea. The BST rule applies to entire subtrees, not only to immediate children.

For example, root 50, left child 30, and 30's right child 55 is invalid. Although $55 > 30$, 55 still lies in 50's left subtree and must be smaller than 50. A reliable validation method passes an allowed interval down the tree. The left child inherits an upper limit from the parent; the right child inherits a lower limit.

5.2 Searching as repeated elimination

To search for 60 in the pictured tree:

1. Compare with 50. Since $60 > 50$, all of the left subtree can be ignored.
2. Compare with 70. Since $60 < 70$, move left.
3. Compare with 60. The key is found.

To search for 65, follow 50, 70, 60, then try the right child of 60. It is empty, so 65 is absent. An unsuccessful search ends at a missing child position, not at an arbitrary leaf merely because it is a leaf.

Search takes time proportional to the length of one root-to-node or root-to-empty-position path: $O(h + 1)$. A well-shaped tree gives $O(\log n)$ worst-case search for that shape; a chain gives $O(n)$.

Algorithm: Search in a BST

Input: a root and a target key. **Output:** the matching node, or “not found.”

Step 1. Start. Set the current position to the root.

Step 2. If the current position is empty, record “not found” and go to Step 7.

Step 3. If the target equals the current key, record the current node as the result and go to Step 7.

Step 4. If the target is smaller, move the current position to the left child and go to Step 6.

Step 5. Otherwise, move the current position to the right child.

Step 6. Repeat from Step 2.

Step 7. End. Return the recorded result.

```
Node *bst_search(Node *root, int key) {
    while (root != NULL) {
        if (key == root->key) return root;
        if (key < root->key) root = root->left;
        else                root = root->right;
    }
    return NULL;
}
```

This function changes only its local reference to the current position. It does not move any tree node. The loop corresponds to repeating Step 2 after each child choice.

5.3 Insertion: search until the new position is found

Start at the root and compare the new key as if searching for it. When the required child position is empty, attach the new node there. An ordinary BST insertion makes the new node a leaf. If the tree is empty, the new node becomes the root.

Worked example: building a BST

Insert 50, 30, 70, 20, 40, 60, 80 in that order.

Key	Comparisons	Final position
50	Tree is empty	Root
30	$30 < 50$	Left of 50
70	$70 > 50$	Right of 50
20	$20 < 50$, $20 < 30$	Left of 30
40	$40 < 50$, $40 > 30$	Right of 30
60	$60 > 50$, $60 < 70$	Left of 70
80	$80 > 50$, $80 > 70$	Right of 70

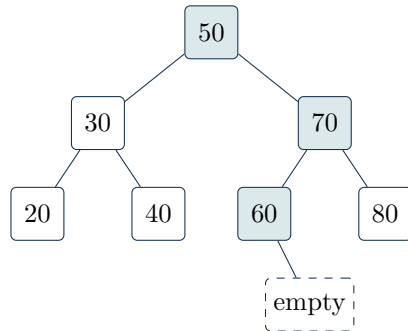
The result is the preceding diagram. Inserting 65 follows 50, 70, 60 and adds 65 as the right child of 60.

5.4 Insertion slowed down: finding a place versus changing a link

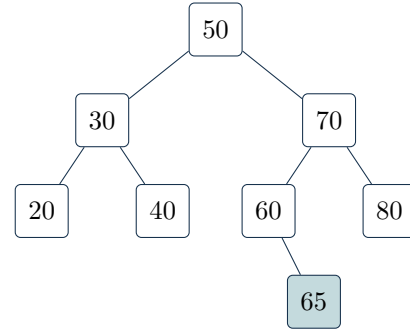
The search phase does not rearrange existing nodes. It narrows the allowed range until a missing child position is reached. Inserting 65 into the seven-node example makes these decisions:

Current node	Comparison	Next move	Allowed range after the move
50	$65 > 50$	Right	Greater than 50
70	$65 < 70$	Left	Between 50 and 70
60	$65 > 60$	Right	Between 60 and 70
Empty	Position found	Attach 65 here	65 satisfies all bounds

Before: follow the highlighted path



After: attach one new leaf



Only 60's right-child reference changes from empty to the new node. The new node has two empty child references. The left subtree under 30 and the node 80 do not need to be inspected.

Algorithm: Insert a distinct key into an ordinary BST

Input: a root and a key. **Output:** the possibly updated root. For these snippets, inserting an existing key leaves the tree unchanged.

- Step 1. Start.** Receive the root and the key.
- Step 2.** If the tree is empty, create a leaf containing the key, make it the root, and go to Step 9.
- Step 3.** Set the current node to the root.
- Step 4.** If the key equals the current key, leave the tree unchanged and go to Step 9.
- Step 5.** Choose the left-child position if the key is smaller; otherwise choose the right-child position.
- Step 6.** If the chosen position contains a node, move to it and repeat from Step 4.
- Step 7.** Create a new node with the key and two empty child references.
- Step 8.** Attach it at the chosen empty child position of the current node.
- Step 9. End.** Return the root of the resulting tree.

5.5 Short C insertion: returning the root of each subtree

The following compact implementation follows the same choices using subtree calls. A call returns the root of its resulting subtree. Assigning that return value to the corresponding child reference is how the new node becomes attached.

```
Node *bst_insert(Node *root, int key) {
    if (root == NULL) return new_node(key);
    if (key < root->key)
```

```

    root->left = bst_insert(root->left, key);
    else if (key > root->key)
        root->right = bst_insert(root->right, key);
    return root;          /* Equal keys make no change. */
}

```

How the 65 example returns. The empty right-subtree call below 60 creates 65 and returns its reference. The paused call at 60 stores that reference as its right child and returns 60. The calls at 70 and 50 then return their own roots. These latter references are the same as before; the attachment happened at the missing position.

At the outermost call, keep the returned root using `root = bst_insert(root, 65);`. This matters especially when the original tree was empty. The implementation assumes the supplied tree already satisfies the BST rule.

5.6 Insertion order changes the shape

Inserting 10, 20, 30, 40, 50 into an ordinary BST produces a right chain of height 4. Inserting the same set in the order 30, 20, 40, 10, 50 produces height 2. Both satisfy the search rule and have identical inorder traversals.

Building a BST by repeated insertion can therefore take $\Theta(n^2)$ time in the worst case. The successive insertions in a chain perform work proportional to $0, 1, 2, \dots, n-1$. Under a uniformly random insertion-order assumption for distinct keys, the expected total is $O(n \log n)$. An ordinary BST does not provide a balancing guarantee.

5.7 Minimum, maximum, predecessor, and successor

Minimum: start at the root and keep moving left until no left child exists.

Maximum: keep moving right until no right child exists.

The **inorder successor** of a node is the next larger key. If the node has a right subtree, its successor is the minimum in that subtree. Otherwise, move upward until you first reach an ancestor whose left subtree contains the original node. If there is no such ancestor, the node is already the maximum and has no successor.

The **inorder predecessor** is the next smaller key. If a left subtree exists, take its maximum. Otherwise use the first relevant ancestor reached from its right side. Without parent links, the original search path can supply these ancestors.

Algorithm: Find the minimum or maximum in a BST

Step 1. Start. Receive the root and choose minimum or maximum.

Step 2. If the tree is empty, report “no key” and go to Step 6.

Step 3. Make the root the current node.

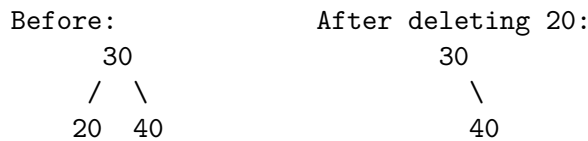
Step 4. For a minimum, move left while a left child exists. For a maximum, move right while a right child exists.

Step 5. Record the current node as the result when the required child no longer exists.

Step 6. End. Return the result or the empty-tree report.

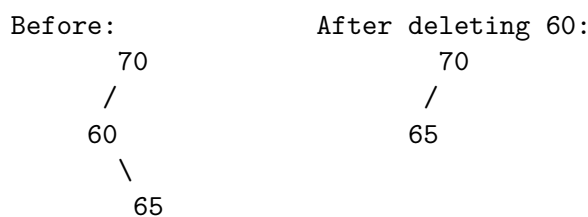
5.8 Deletion case 1: the node is a leaf

If deleting 20 from the original seven-node tree, simply remove the connection from 30 to 20. No descendants need to be reattached. The key's parent now has an empty left-child position.



5.9 Deletion case 2: the node has one child

The surviving child takes the deleted node's place. In effect, bypass the removed node.



The parent link that pointed to 60 now points to 65. If the removed node was the root, the root reference must point to its surviving child. If it was the only node, the root becomes empty.

5.10 Deletion case 3: the node has two children

You cannot simply attach both children to the same child position of the parent. Instead choose an adjacent key in sorted order: the inorder successor or predecessor.

Using the successor:

1. Find the smallest key in the node's right subtree.
2. Replace the target's stored key and associated record with that successor's key and record, or structurally move the successor node into the position.
3. Delete the successor from its original position.

The successor cannot have a left child, because then it would not be the minimum. It therefore has at most one child, so its removal reduces to case 1 or case 2. A predecessor replacement works symmetrically.

Worked example: deleting the root 50

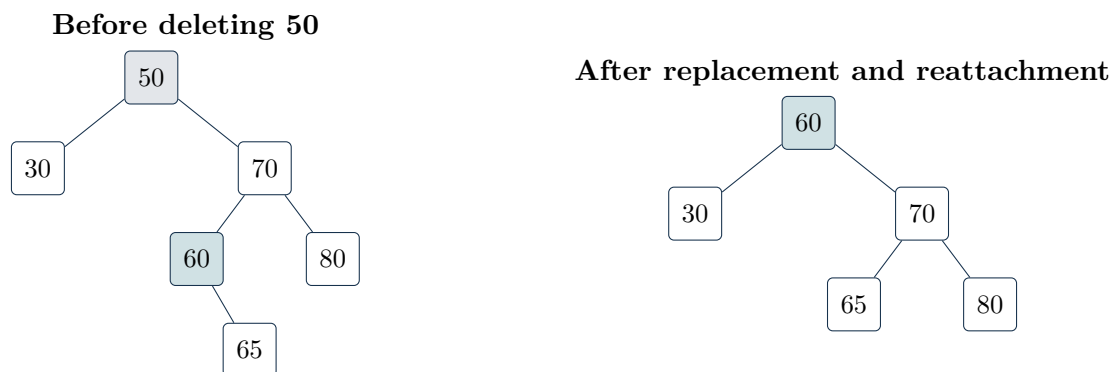
Use a fresh copy of the original seven-node tree. Its root's successor is 60, the leftmost node of the right subtree.



Replace 50 by 60 and remove the old leaf containing 60. Inorder changes from 20, 30, 40, 50, 60, 70, 80 to 20, 30, 40, 60, 70, 80. All other keys remain.

Worked example: the successor itself has a right child

Suppose the right subtree of 50 is rooted at 70; 70 has left child 60; 60 has right child 65. The successor of 50 is still 60. After moving 60 into 50's position, attach 65 as 70's left child. Forgetting 65 would lose a valid subtree.



Here, 65 is larger than the replacement root 60 but smaller than 70, so its new position remains valid. The subtree under 30 is drawn as a single node to keep the changing portion clear.

Algorithm: Delete a key from an ordinary BST

Input: the root of the current subtree and a target key. **Output:** that subtree's root after deletion. A missing target leaves the tree unchanged.

- Step 1. Start.** Receive the current subtree root and target.
- Step 2.** If the subtree is empty, return an empty subtree and end this call.
- Step 3.** If the target is smaller, apply this algorithm to the left subtree, attach its returned root as the left child, and go to Step 9.
- Step 4.** If the target is larger, apply this algorithm to the right subtree, attach its returned root as the right child, and go to Step 9.
- Step 5.** The keys now match. If the node has no children, release that node and return an empty subtree; end this call.
- Step 6.** If it has exactly one child, save that child's reference, release the target node, and return the saved child as the new subtree root; end this call.
- Step 7.** If it has two children, find the minimum node in its right subtree. Copy that successor's key and associated record into the target node.
- Step 8.** Apply this deletion algorithm to the right subtree using the successor's key, and attach the returned right-subtree root.
- Step 9. End this call.** Return the current subtree root. The outermost caller saves the returned overall root.

5.11 Short C deletion: the three cases in one function

The code combines the leaf and one-child cases: when a child reference is empty, return the other child reference, which may also be empty.

```
Node *bst_delete(Node *root, int key) {
    if (root == NULL) return NULL;
    if (key < root->key) {
        root->left = bst_delete(root->left, key);
    } else if (key > root->key) {
        root->right = bst_delete(root->right, key);
    } else {
        if (root->left == NULL || root->right == NULL) {
            Node *child = root->left ? root->left : root->right;
            free(root);
            return child;
        }
        Node *next = root->right;
        while (next->left != NULL) next = next->left;
        root->key = next->key;
        root->right = bst_delete(root->right, next->key);
    }
    return root;
}
```

Read the important lines. Save the surviving child's reference before freeing the removed node. For two children, the loop finds the successor; the last subtree call removes its old occurrence. Use `root = bst_delete(root, key);` so deleting the overall root is handled correctly. This is ordinary BST deletion; it does not perform AVL repairs.

Releasing an entire tree uses postorder. Its children must be released before their parent, because the parent stores the references needed to find them.

```
void destroy_tree(Node *root) {
    if (root == NULL) return;
    destroy_tree(root->left);
    destroy_tree(root->right);
    free(root);
}
```

After calling this on the whole tree, set the caller's root reference to NULL; the old nodes no longer exist. Notice that only the visit action changed from the earlier postorder example: release the node instead of printing its key.

5.12 Why successor replacement preserves the order

The successor is larger than the old root and every key in its left subtree, because it comes from the right subtree. It is no larger than any key remaining in that right subtree, and with distinct keys it is strictly smaller than all of them. Therefore it fits between the left and right groups. Removing its old occurrence leaves exactly one copy.

5.13 BST operation costs

Searching, inserting, deleting, finding the minimum, and finding the maximum all have $O(h + 1)$ worst-case time for a tree of height h . A deletion with two children still follows only a bounded

number of paths whose combined length is $O(h + 1)$; it does not require a full traversal.

An iterative search can use $O(1)$ extra space. A recursive version uses $O(h + 1)$ reminder space. Inorder traversal always takes $\Theta(n)$ time and produces the sorted sequence for distinct keys. The tree itself occupies $\Theta(n)$ space.

Common mistake. Replacing a two-child node with an arbitrary descendant can break the BST rule. Use the successor or predecessor and repair the original location.

6 AVL trees: keeping a search tree short

6.1 The problem AVL trees solve

A BST can become a long chain even though its ordering rule is correct. An **AVL tree** is a BST that also keeps the heights of each node's left and right subtrees close. It preserves fast searching even when keys arrive in an inconvenient order.

For a node v , define

$$\text{BF}(v) = h(\text{left subtree}) - h(\text{right subtree}).$$

BF stands for **balance factor**. The AVL requirement is

$$\text{BF}(v) \in \{-1, 0, +1\} \text{ for every node } v.$$

Positive means taller on the left. Negative means taller on the right. A leaf has two empty subtrees, so its balance factor is $-1 - (-1) = 0$.

The height of a node is $1 + \max(h_L, h_R)$. With empty height -1 , this gives leaf height 0 automatically.

6.2 Balance must hold everywhere

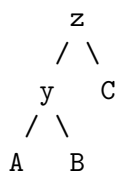
A root with equally tall subtrees can still have an unbalanced descendant. Check all relevant nodes, not just the root. AVL does not require a perfect tree, identical subtree sizes, or the complete-tree shape used by heaps. It compares heights locally.

6.3 What a rotation actually does

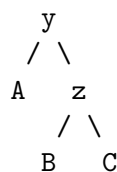
A **rotation** changes a few parent-child connections so that a nearby node becomes the local root. It preserves the left-to-right sorted order. It does not sort a scrambled tree, and it does not rebuild the entire tree.

Right rotation around z:

Before:



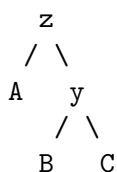
After:



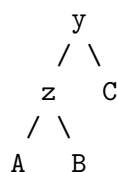
The left child y rises. z moves down to the right. y 's old right subtree B becomes z 's left subtree. The value groups obey $A < y < B < z < C$, where this notation means every key in each group lies in the indicated range. The inorder arrangement is unchanged.

Left rotation around z:

Before:



After:



The right child y rises. z moves down to the left. B transfers to z 's right side. A single rotation takes $O(1)$ time when the relevant links and stored heights are available. Heights of the affected nodes must be updated, lower node before new local root.

Common mistake. The middle subtree B is transferred, not discarded. Draw it explicitly when a rotation includes non-leaf nodes.

Algorithm: Right rotation

Input: a subtree root z with an existing left child. **Output:** the new local root. The input is already ordered as a BST.

- Step 1. Start.** Receive z and remember how its parent refers to this subtree, if performing the links directly.
- Step 2.** Name z 's left child y .
- Step 3.** Save y 's right-subtree reference as the middle subtree B .
- Step 4.** Make z the right child of y .
- Step 5.** Make B the left subtree of z .
- Step 6.** Update z 's height, then update y 's height, using their new children.
- Step 7.** Reconnect the former parent to y , or make y the overall root. In a return-based implementation, let the caller perform this attachment.
- Step 8. End.** Return y as the new subtree root.

Algorithm: Left rotation

Input: a subtree root z with an existing right child. **Output:** the new local root.

- Step 1. Start.** Receive z and remember its connection from its parent, if updating links directly.
- Step 2.** Name z 's right child y .
- Step 3.** Save y 's left-subtree reference as the middle subtree B .
- Step 4.** Make z the left child of y .
- Step 5.** Make B the right subtree of z .
- Step 6.** Update z 's height first, then y 's height.
- Step 7.** Reconnect the former parent to y , or make y the overall root; alternatively, return y for the caller to attach.
- Step 8. End.** Return y as the new subtree root.

6.4 C rotations: a small number of changed references

These helpers use the node's stored height. Reading it is constant time; recomputing an entire subtree's height at every ancestor would lose the intended update efficiency.

```

int node_height(const Node *p) {
    return p == NULL ? -1 : p->height;
}

void update_height(Node *p) {
    int left = node_height(p->left);
    int right = node_height(p->right);
    p->height = 1 + (left > right ? left : right);
}

Node *rotate_right(Node *z) {
    Node *y = z->left;
    Node *middle = y->right;
    y->right = z;
    z->left = middle;
    update_height(z);
    update_height(y);
    return y;
}

Node *rotate_left(Node *z) {
    Node *y = z->right;
    Node *middle = y->left;
    y->left = z;
    z->right = middle;
    update_height(z);
    update_height(y);
    return y;
}

```

Each rotation assumes the required child exists. The caller chooses a rotation only after detecting the corresponding imbalance. The middle subtree is saved before the old child reference is overwritten. The caller must keep the returned root, because *z* is no longer the local root.

6.5 The four insertion cases

After a BST insertion, walk upward from the inserted node and find the **first unbalanced ancestor**, called *z*. Let *y* be *z*'s child on the path toward the inserted node. The first two directions from *z* along this path name the case.

Case	Path from <i>z</i>	Shape	Repair
LL	Left, then left	Outside bend	Right rotation at <i>z</i>
RR	Right, then right	Outside bend	Left rotation at <i>z</i>
LR	Left, then right	Inside bend	Left at <i>y</i> , then right at <i>z</i>
RL	Right, then left	Inside bend	Right at <i>y</i> , then left at <i>z</i>

“LL” names the insertion path, not the direction of the corrective rotation.

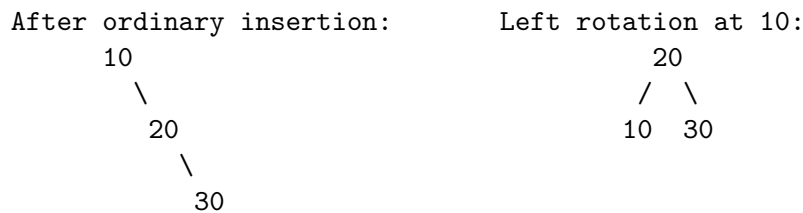
Worked example: LL: insert 30, 20, 10

After ordinary insertion: Right rotation at 30:



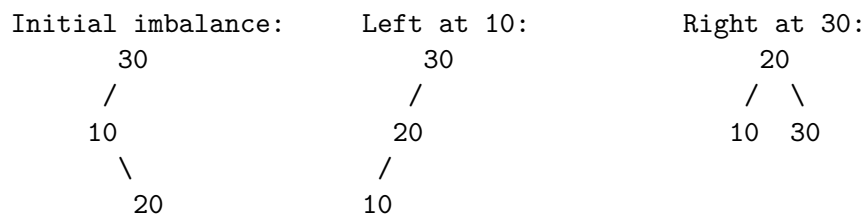
At 30, the left height is 1 and the right height is -1 , so BF is $+2$. The path is left-left. After rotation, the new local root is 20 and all three balance factors are 0.

Worked example: RR: insert 10, 20, 30



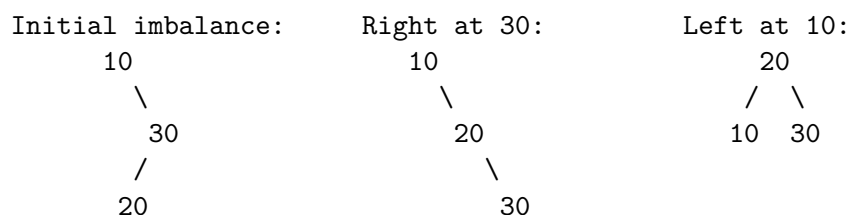
At 10, BF is -2 and the path is right-right. The corrective rotation is left.

Worked example: LR: insert 30, 10, 20



The first rotation straightens the inside bend into an LL shape. The second restores balance. Trying only a right rotation at the original root does not fix this case.

Worked example: RL: insert 10, 30, 20



The first rotation straightens the inside bend into an RR shape. The second completes the repair.

6.6 A complete AVL insertion routine in words

Algorithm: AVL insertion

Input: a valid AVL tree and a key. **Output:** a valid AVL tree containing that key. Existing keys are left unchanged.

- Step 1. Start.** Receive the root and key.
- Step 2.** Find the ordinary BST insertion position. If the key already exists, keep the original root and go to Step 9.
- Step 3.** Attach the new leaf with height 0. If it becomes the root of an empty tree, go to Step 9.
- Step 4.** Move to the next ancestor above the insertion position, working upward. If no ancestor remains, go to Step 9.
- Step 5.** Update that ancestor's height and compute left height minus right height.
- Step 6.** If the factor is -1 , 0 , or $+1$, repeat from Step 4 with the next ancestor.
- Step 7.** At the first unbalanced ancestor z , inspect the first two path directions toward the new key. LL needs right at z ; RR needs left at z ; LR needs left at z 's left child then right at z ; RL needs right at z 's right child then left at z .
- Step 8.** Perform the required rotation or rotations, update affected heights from lower nodes upward, and attach the returned local root to its former parent or to the overall root reference.
- Step 9. End.** Return the resulting overall root. In insertion, one repair site suffices, even if it uses two rotations.

After insertion repair, the repaired subtree has the height it had before the insertion, so ancestors above it do not acquire a new imbalance from this insertion. Heights may still be tracked as part of the chosen bookkeeping method.

6.7 Short C AVL insertion: BST choices plus local repair

This implementation starts with an AVL tree whose stored heights are correct. The first part descends to the insertion point. As each call returns, it refreshes the current height and repairs a violation if needed.

```
Node *avl_insert(Node *root, int key) {
    if (root == NULL) return new_node(key);
    if (key < root->key)
        root->left = avl_insert(root->left, key);
    else if (key > root->key)
        root->right = avl_insert(root->right, key);
    else return root;

    update_height(root);
    int balance = node_height(root->left) - node_height(root->right);
    if (balance > 1) {
        if (key > root->left->key) /* LR: straighten first. */
            root->left = rotate_left(root->left);
        return rotate_right(root); /* LL or completed LR. */
    }
    if (balance < -1) {
        if (key < root->right->key) /* RL: straighten first. */
            root->right = rotate_right(root->right);
        return rotate_left(root); /* RR or completed RL. */
    }
    return root;
}
```

Call it with `root = avl_insert(root, key);`. The reference returned by the final rotation may differ from the incoming root. There is no full-tree search for an imbalance: only nodes on the insertion path have acquired changed subtree heights.

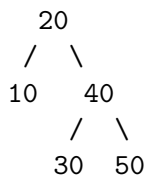
Why this remains compact. The four repair cases share two final rotations. The inner tests perform the first rotation only for an inside bend. The named LL, RR, LR, and RL diagrams remain the guide to understanding these lines.

Worked example: a longer insertion sequence

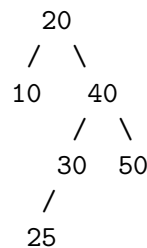
Insert 10, 20, 30, 40, 50, 25 into an initially empty AVL tree.

1. 10 becomes the root; 20 becomes its right child. The root has BF -1 , so no repair is needed.
2. Insert 30. Node 10 has an RR imbalance. Rotate left at 10. The root is now 20, with children 10 and 30.
3. Insert 40 as the right child of 30. All nodes remain within the allowed balance range.
4. Insert 50 as the right child of 40. The first imbalance is at 30, an RR case. Rotate left at 30.

After inserting 50:

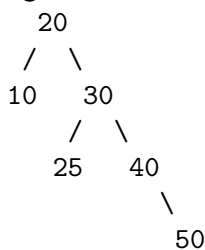


After inserting 25, before repair:

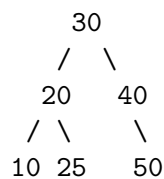


At 20, left height is 0 and right height is 2, giving BF -2 . From 20, the new key lies right, then left: an RL case. Rotate right at 40, then left at 20.

After right rotation at 40:



After left rotation at 20:



The final inorder is 10, 20, 25, 30, 40, 50. Leaves have height 0; nodes 20 and 40 have height 1; root 30 has height 2. Their balance factors are 0, -1 , and 0 respectively.

6.8 Why AVL height is logarithmic

Let $M(h)$ be the minimum number of nodes in an AVL tree of height h . To make the tree as tall as possible using few nodes, choose child heights $h-1$ and $h-2$. A larger difference would violate the AVL rule. Thus

$$M(-1) = 0, \quad M(0) = 1, \quad M(h) = 1 + M(h-1) + M(h-2) \quad (h \geq 1).$$

The first counts are 1, 2, 4, 7, 12, 20, 33 for heights 0 through 6. These counts grow exponentially with height, so height grows only logarithmically with node count. Therefore AVL search and insertion have $O(\log n)$ worst-case time. The tree occupies $\Theta(n)$ storage, including a small amount of height or balance information per node.

6.9 AVL deletion: a short boundary note

The supplied topic list specifically names AVL rotations and insertion. For context, AVL deletion first performs BST deletion and then repairs balance while moving upward. Unlike insertion, deletion can reduce the repaired subtree's height and require repairs at multiple ancestors. A heavy child's balance factor can be 0 during deletion, so insertion-path shortcuts should not be copied blindly. AVL deletion remains $O(\log n)$ with correct height maintenance; a full deletion procedure is beyond the requested AVL emphasis.

7 Binary heaps and heapify

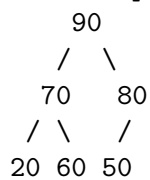
7.1 The two separate rules of a binary heap

A **binary heap** is a complete binary tree with a parent-child priority rule. Both the shape and the value rule matter.

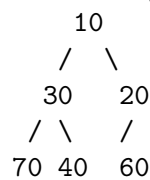
- In a **max-heap**, every parent is at least as large as either child. The largest value is at the root.
- In a **min-heap**, every parent is at most as large as either child. The smallest value is at the root.

Equality is allowed. The rule applies to every parent-child connection. It therefore also ensures that a max-heap root is at least as large as every descendant, and a min-heap root is at most as large as every descendant.

Valid max-heap:



Valid min-heap:



Key idea. A heap controls parent-child priority. It does not require the whole left subtree to be smaller than the whole right subtree.

The max-heap above has array $[90, 70, 80, 20, 60, 50]$. It is neither increasing nor decreasing. Its inorder is $20, 70, 60, 90, 50, 80$, which is not sorted. The largest value is easy to find, but locating an arbitrary value may require examining all n entries.

7.2 Why the complete shape matters

A complete tree has height $\lfloor \log_2 n \rfloor$, so moving from a leaf to the root or from the root to a leaf takes only $O(\log n)$ steps. It also occupies consecutive array positions without structural gaps. A heap is usually stored this way rather than as separately linked nodes.

Using one-based positions, a heap's leaves are at positions

$$\lfloor n/2 \rfloor + 1, \dots, n.$$

The last internal node is at $\lfloor n/2 \rfloor$. For example, when $n = 6$, internal positions are 1, 2, 3 and leaf positions are 4, 5, 6.

7.3 Heapify: repairing one local violation

The word **heapify** is used in two related ways in textbooks: repairing one node downward, and building an entire heap. Here, **sift down** means one-node repair, and **build-heap** means repairing the whole array from the bottom upward.

For max-heap sift down, assume the node's left and right subtrees already satisfy the max-heap rule. Only the current node may be too small.

Algorithm: Max-heap sift down

Input: an active heap region and a current position whose child subtrees are already max-heaps.

Output: a repaired subtree.

Step 1. Start. Receive the current position and the active heap size.

Step 2. If the current position has no child inside the active region, go to Step 7.

Step 3. Choose the larger existing child. With equal children, either choice is valid; use the left one consistently here.

Step 4. If the current value is at least as large as that child's value, go to Step 7.

Step 5. Exchange the current value with the chosen child's value.

Step 6. Make that child's position the current position and repeat from Step 2.

Step 7. End. The subtree rooted at the original position obeys the max-heap rule.

For a min-heap, exchange with the smaller child when the current value is too large.

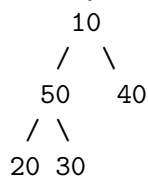
Worked example: why choosing the larger child is necessary

Suppose a max-heap repair reaches parent 10 with children 30 and 50. Swapping with 30 leaves parent 30 above child 50, so the parent is still invalid. Swapping with 50 puts 50 above both 30 and 10, fixing that local relationship. Any remaining problem is lower down where 10 moved.

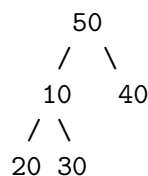
7.4 Sift down, step by step

Start with [10, 50, 40, 20, 30]. The subtrees rooted at 50 and 40 are already max-heaps.

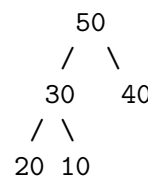
Initially:



First exchange:



Second exchange:



Choose 50 over 40 and exchange with 10. Now 10 has children 20 and 30, so exchange with 30. The final array is [50, 30, 40, 20, 10].

Sift down may stop immediately, taking $\Theta(1)$ time, or travel the heap's height, taking $O(\log n)$ worst-case time. An iterative version needs $O(1)$ auxiliary space.

7.5 Building a heap from an unordered array

All leaves already satisfy the heap rule because they have no children. Begin with the last internal node, then work backward toward the root. Sift down each node. When you reach a parent, its child subtrees have already been repaired.

Algorithm: Build a max-heap bottom-up

Input: n values in one-based positions 1 through n . **Output:** the same values arranged as a max-heap.

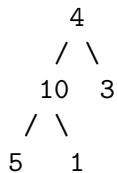
- Step 1. Start.** Receive the array and its item count n .
- Step 2.** If n is 0 or 1, no repair is needed; go to Step 7.
- Step 3.** Set the current parent position to $\lfloor n/2 \rfloor$, the last internal node.
- Step 4.** Apply max-heap sift down at that position, using all n items as the active region.
- Step 5.** Move the current parent position one place to the left in the array.
- Step 6.** If that position is at least 1, repeat from Step 4.
- Step 7. End.** The whole array satisfies the max-heap rule.

Worked example: bottom-up build of a max-heap

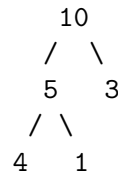
Start with $[4, 10, 3, 5, 1]$, with positions 1 through 5. The last internal position is $\lfloor 5/2 \rfloor = 2$.

1. At position 2, value 10 has children 5 and 1. It is already valid. The array stays $[4, 10, 3, 5, 1]$.
2. At position 1, value 4 has children 10 and 3. Exchange 4 with 10: $[10, 4, 3, 5, 1]$.
3. Continue from position 2. Value 4 has children 5 and 1. Exchange with 5: $[10, 5, 3, 4, 1]$.

Before build:



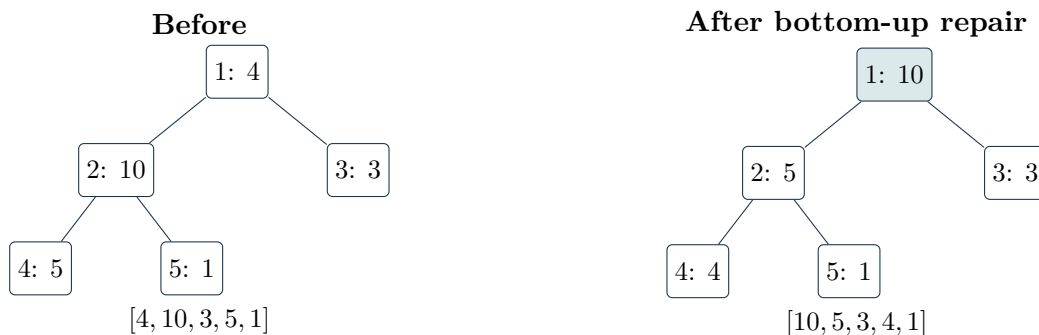
After build:



The final heap is not a sorted array. It simply satisfies the two heap rules.

7.6 A graphical link between the array and its tree

The labels below read **position: value**. This makes it possible to see where each exchange in the previous example happened. These pictures use one-based positions.



Array positions determine the edges. Repair exchanges *values*; it does not change the complete-tree shape or the position formulas.

7.7 Short C heap repair and construction

Index convention changes here: C uses indices 0 through $n - 1$. Children of index i are $2i + 1$ and $2i + 2$. The parameter n is the *active item count*, not the last index. These snippets use a nonnegative `int` count and an array containing at least that many entries.

```
void sift_down(int a[], int n, int i) {
    while (i < n / 2) {           /* i has a left child. */
        int child = 2 * i + 1;
        if (child + 1 < n && a[child + 1] > a[child])
            ++child;
        if (a[i] >= a[child]) break;
        int temp = a[i];
        a[i] = a[child];
        a[child] = temp;
        i = child;
    }
}

void build_max_heap(int a[], int n) {
    for (int i = n / 2 - 1; i >= 0; --i)
        sift_down(a, n, i);
}
```

Read the first loop carefully. The current position can have children only when it is an internal node. Test for the right child before reading it. Move down only after an exchange. The build loop starts at the last internal index and moves toward index 0. For $n = 0$ or 1, that build loop performs no repairs.

7.8 Why bottom-up build-heap is linear

A loose argument says there are n repairs and each costs at most $O(\log n)$, giving $O(n \log n)$. That upper bound is true but misses how little work most nodes do.

About half the nodes are leaves and need no downward movement. About a quarter can move down at most one level. About an eighth can move down at most two levels. Only a few nodes are near the root. The work is bounded by a constant multiple of

$$n \left(\frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \cdots \right) = O(n).$$

The weighted fraction sum converges to a constant. More precise counting handles rounding at the last level without changing the result. Processing the internal nodes also requires linear work overall, so standard bottom-up build-heap takes $\Theta(n)$ time.

Common mistake. One sift-down operation is $O(\log n)$ in the worst case. Building a heap bottom-up is $\Theta(n)$. These are different tasks.

Building by inserting items one at a time is another valid approach, but its worst-case cost is $O(n \log n)$. Do not use that cost for the bottom-up method.

7.9 Insertion into a heap: sift up

Put the new value in the next free array position to preserve completeness. Compare it with its parent. In a max-heap, exchange while the new value is larger; in a min-heap, exchange while it is smaller. Continue upward until the rule holds or the root is reached.

Algorithm: Insert into a max-heap

Input: a valid max-heap and a new value. **Output:** a max-heap containing the new value. The array must have space for one more item, or be enlarged first.

- Step 1. Start.** Receive the heap and new value.
- Step 2.** Put the new value in the next unused array position and increase the active item count by one.
- Step 3.** Make that last position the current position.
- Step 4.** If the current position is the root, go to Step 8.
- Step 5.** Find its parent position. In the one-based convention, this is $\lfloor i/2 \rfloor$.
- Step 6.** If the parent value is at least as large as the current value, go to Step 8.
- Step 7.** Exchange the two values, move to the parent position, and repeat from Step 4.
- Step 8. End.** Return the updated heap and item count.

Worked example: insert 60 into a max-heap

Start with [50, 30, 40, 20, 10].

Append: [50, 30, 40, 20, 10, 60]

Exchange with parent 40: [50, 30, 60, 20, 10, 40]

Exchange with parent 50: [60, 30, 50, 20, 10, 40].

The new item begins at position 6, moves to position 3, then position 1. Time is $O(\log n)$ in the worst case, excluding an occasional array-resizing cost in a dynamic-storage implementation.

7.10 Removing the root

To remove the maximum from a max-heap:

Algorithm: Remove the maximum from a max-heap

Input: a max-heap. **Output:** the removed value and the smaller heap, or an empty-heap report.

- Step 1. Start.** Receive the heap and active item count.
- Step 2.** If the heap is empty, report that no item can be removed and go to Step 8.
- Step 3.** Save the root's value as the value to return.
- Step 4.** If there is only one item, make the heap empty and go to Step 8.
- Step 5.** Move the last item's value to the root position.
- Step 6.** Reduce the active item count by one, excluding the old last position.
- Step 7.** Apply sift down at the root using the smaller active region.
- Step 8. End.** Return the saved maximum and new count, or the empty-heap report.

Using the last value preserves the complete shape. Removing the minimum from a min-heap uses the same idea with reversed comparisons.

Worked example: remove the maximum 60

Start with $[60, 30, 50, 20, 10, 40]$. Move 40 to the root and remove the last position: $[40, 30, 50, 20, 10]$. The larger child is 50, so exchange: $[50, 30, 40, 20, 10]$. Return 60. The operation takes $O(\log n)$ worst-case time.

For a one-item heap, removal simply makes it empty. An empty heap has no minimum or maximum; an operation must report that there is no item to remove.

7.11 Heap operation summary and applications

Operation	Typical binary-heap cost
Read minimum in min-heap / maximum in max-heap	$\Theta(1)$
Insert or remove root	$O(\log n)$ worst case
Bottom-up build from n values	$\Theta(n)$
Search for an arbitrary key	$O(n)$ worst case
Change priority at a known position	$O(\log n)$ worst case
Stored items	$\Theta(n)$ space

A **priority queue** serves the most urgent item first rather than the earliest arrival. A heap is one way to implement it. Examples include choosing the next scheduled event, handling urgent jobs, maintaining the best few candidates, and selecting the two least frequent groups in Huffman construction. Whether high or low numbers mean “urgent” determines max-heap versus min-heap.

If the position of an item whose priority changes is unknown, finding it can take $O(n)$ first. The $O(\log n)$ update assumes its position is already known or can be retrieved separately.

8 Heap sort: sorting by repeatedly removing the largest

8.1 The main idea

For ascending order, build a max-heap. The root contains the largest unsorted value. Exchange it with the last value in the active heap. That largest value is now in its final sorted position. Reduce the active heap size by one, repair its root, and repeat.

The array is divided into two regions:

[active max-heap: still being worked on | sorted suffix]

The **suffix** is simply the region at the right end. Its values are already in ascending order, and every value there is at least as large as every value still in the heap.

Algorithm: Heap sort into ascending order

Input: an array of n values. **Output:** the same array sorted from smallest to largest.

Step 1. Start. Receive the array and item count.

Step 2. Build a max-heap from the entire array using bottom-up construction.

Step 3. Make all n positions the active heap region. Initially, the sorted suffix is empty.

Step 4. If the active region has at most one item, go to Step 8.

Step 5. Exchange the root value with the value in the last active position.

Step 6. Shorten the active region by one position. The excluded maximum now belongs to the sorted suffix.

Step 7. Sift down at the root within the active region only, then repeat from Step 4.

Step 8. End. The complete array is in ascending order.

8.2 A complete trace

Sort [4, 10, 3, 5, 1] in ascending order. Bottom-up build gives [10, 5, 3, 4, 1].

Action	Array: active region sorted region	Explanation
Build max-heap	[10, 5, 3, 4, 1]	Largest value is 10
Move 10 to the end	[1, 5, 3, 4 10]	Active size becomes 4
Repair active heap	[5, 4, 3, 1 10]	1 moves below 5 and 4
Move 5 to the end	[1, 4, 3 5, 10]	Active size becomes 3
Repair active heap	[4, 1, 3 5, 10]	Exchange 1 and 4
Move 4 to the end	[3, 1 4, 5, 10]	Active size becomes 2
Repair active heap	[3, 1 4, 5, 10]	Already valid
Move 3 to the end	[1 3, 4, 5, 10]	One active value remains
Finished	[1, 3, 4, 5, 10]	All values sorted

After each exchange, the old root position may violate the heap rule. Sift down only inside the active region. Comparing with values already in the sorted suffix would spoil the progress.

8.3 Short C heap sort: shrinking the active region

This uses `build_max_heap` and `sift_down` from the preceding section, with zero-based indices.

```
void heap_sort(int a[], int n) {
    build_max_heap(a, n);
    for (int end = n - 1; end > 0; --end) {
        int temp = a[0];
        a[0] = a[end];
        a[end] = temp;
        sift_down(a, end, 0);
    }
}
```

Why the repair receives `end`. After the exchange, index `end` is sorted and must be excluded. Remaining active indices are 0 through `end - 1`, which contain exactly `end` items. Passing `end + 1` would mistakenly include the newly sorted maximum.

8.4 Why it works

Before each removal, the active region is a max-heap. Its root is therefore the largest remaining value. Moving it to the rightmost active position places it immediately before the values that were removed earlier; those earlier values are at least as large. The growing suffix stays sorted. Repair restores the heap rule for the next round. When only one active item remains, there is nothing left to compare or place.

This kind of statement that remains true before and after each round is called an **invariant**. It is a useful way to explain correctness without code.

8.5 Time and memory

Building the heap costs $\Theta(n)$. There are $n - 1$ root exchanges, and each repair costs at most $O(\log n)$. Total worst-case time is $\Theta(n \log n)$. In the usual distinct-key setting, standard heap sort also has $\Theta(n \log n)$ best- and average-case time. With equal keys and a sift-down routine that stops immediately when no exchange is needed, special inputs such as all-equal values can take linear time. The important guarantee is the $O(n \log n)$ worst-case bound.

An array-based version with iterative sift down uses $O(1)$ auxiliary storage: it rearranges the existing array. A recursive sift-down implementation may use $O(\log n)$ reminder space. The array containing the input itself uses $\Theta(n)$ space and is not counted as auxiliary storage.

8.6 Stability and direction of sorting

A sorting method is **stable** if equal-key records keep their original relative order. Ordinary heap sort is not stable because distant exchanges can move equal records past one another.

For example, let 2_a and 2_b have the same key 2 but different identities. Begin with $[2_a, 2_b, 1]$. It is already a max-heap. Exchange the root with 1, then repair by moving 2_b up, giving $[2_b, 1 \mid 2_a]$. The next exchange produces $[1, 2_b, 2_a]$. The two equal records have reversed order.

For descending order using the same end-placement scheme, use a min-heap. Alternatively, repeatedly removing the minimum and *appending to a separate output sequence* gives ascending order. Always distinguish sorting inside the original array from collecting removed values in another list.

Common mistake. Ascending in-place heap sort uses a max-heap because the largest remaining value is placed at the right end.

9 Huffman trees: shorter codes for frequent symbols

9.1 The problem being solved

Suppose a message contains a small set of symbols, but some symbols occur much more often than others. A **fixed-length code** gives every symbol the same number of bits. A **bit** is a 0 or a 1. With $k \geq 2$ distinct symbols, a fixed-length binary code needs $\lceil \log_2 k \rceil$ bits per symbol.

A **variable-length code** gives different symbols different numbers of bits. Frequent symbols should usually get shorter codes so that the whole message uses fewer bits. However, the combined bit stream must still be readable without guessing where one symbol ends and the next begins.

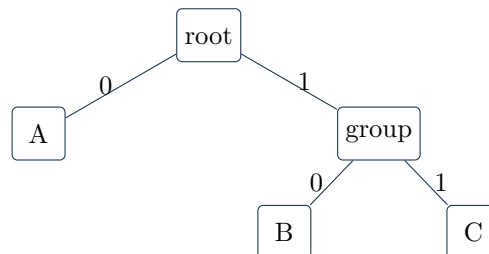
9.2 The prefix-free rule

A code is **prefix-free** if no complete symbol code is the beginning of another symbol's code. For example, $A = 0$, $B = 10$, $C = 11$ is prefix-free. But $A = 0$, $B = 01$ is not, because 0 is the beginning of 01.

Without a suitable rule, ambiguity can occur. With $A = 0$, $B = 01$, $C = 1$, the stream 01 could mean B or A followed by C. Prefix-free coding avoids this by making each leaf reached during decoding mark a complete symbol.

9.3 A code tree

In a binary code tree, each symbol occupies a leaf. Assign 0 to a left edge and 1 to a right edge. A symbol's code is the sequence of edge labels on its root-to-leaf path.



The codes are $A = 0$, $B = 10$, $C = 11$. Leaves cannot be ancestors of other leaves, so one symbol code cannot be the prefix of another. Internal nodes represent groups, not individual symbols to output.

9.4 Huffman's construction rule

A **frequency** is how often a symbol occurs; a weight can also be a supplied relative frequency. Begin with one leaf per positive-frequency symbol. Repeatedly combine the two groups with the smallest weights.

Algorithm: Construct a Huffman tree

Input: distinct symbols and their positive frequencies. **Output:** a Huffman tree, or an empty result if there are no symbols.

- Step 1. Start.** Receive the symbols and frequencies. If given a message instead, count its symbol frequencies first.
- Step 2.** Create one leaf per positive-frequency symbol, storing that symbol and its frequency.
- Step 3.** Put all leaves in a min-priority waiting collection, such as a min-heap.
- Step 4.** If the collection is empty, record an empty result and go to Step 10. If it contains one tree, record that tree as the result and go to Step 10.
- Step 5.** Remove the two trees with the lowest weights from the current collection.
- Step 6.** Create a new parent with weight equal to the sum of those two weights.
- Step 7.** Attach the two selected trees as its left and right subtrees. Use any consistent tie-breaking and left/right convention.
- Step 8.** Put the combined tree back into the collection.
- Step 9.** Repeat from Step 4, including combined trees in every new selection.
- Step 10. End.** Return the remaining tree, or the empty result. Assign its bit codes with the next algorithm.

The collection contains both original leaves and combined groups. Every selection must consider *all* of them. Choosing only from uncombined original symbols is incorrect. A min-heap makes these repeated selections efficient.

Algorithm: Read the codes from a Huffman tree

Input: the completed tree. **Output:** a symbol-to-bit-string codebook.

- Step 1. Start.** Begin at the root with an empty path of bits.
- Step 2.** If the tree is empty, return an empty codebook and end. If its root is the only leaf, choose a single-symbol convention; use code 0 here and end.
- Step 3.** At the current node, if it is a leaf, record the current path as that symbol's code and return to the caller without descending farther.
- Step 4.** For the left child, append 0 to the current path, apply Steps 3–5 to that child, and then restore the path that existed before appending 0.
- Step 5.** For the right child, append 1, apply Steps 3–5 to that child, and then restore the previous path. Return to the caller after both children finish.
- Step 6. End.** When the original root's traversal finishes, all leaf codes have been recorded.

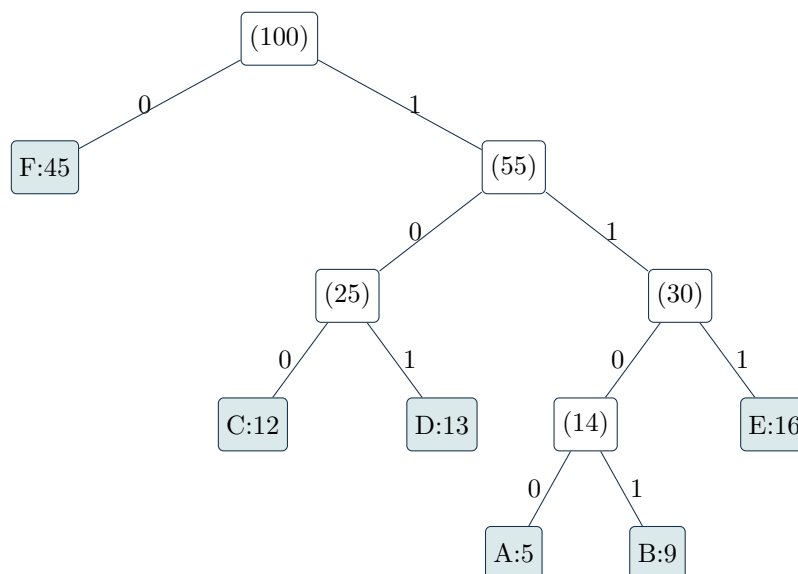
Restoring the path is essential: finishing A's code does not mean B's code begins with all of A's bits. Only their shared ancestor path is shared. This is the same “finish a subtree, return, continue” idea used in traversal.

Worked example: the full six-symbol example

Symbol	A	B	C	D	E	F
Frequency	5	9	12	13	16	45

Total frequency is 100. Start with weights 5, 9, 12, 13, 16, 45.

Round	Merge	Remaining weights, in increasing order
1	$5 + 9 = 14$	12, 13, 14, 16, 45
2	$12 + 13 = 25$	14, 16, 25, 45
3	$14 + 16 = 30$	25, 30, 45
4	$25 + 30 = 55$	45, 55
5	$45 + 55 = 100$	100



Round brackets identify internal weights; a shaded leaf such as A:5 identifies a symbol and its frequency. Read the bits on edges, not the numbers inside the nodes. For A, follow right, right, left, left to obtain 1100. For F, one left edge gives 0.

Symbol	Frequency	Code	Length
F	45	0	1
C	12	100	3
D	13	101	3
A	5	1100	4
B	9	1101	4
E	16	111	3

9.5 Calculating the total cost

The **weighted path length** is the sum of frequency multiplied by leaf depth. Leaf depth equals code length, so it is also the encoded payload size in bits:

$$\begin{aligned}
 \text{Total bits} &= 45(1) + 12(3) + 13(3) + 5(4) + 9(4) + 16(3) \\
 &= 45 + 36 + 39 + 20 + 36 + 48 = 224.
 \end{aligned}$$

The average is $224/100 = 2.24$ bits per symbol. Six symbols need 3 bits each under a fixed-length binary code, giving 300 bits. The Huffman payload saves $300 - 224 = 76$ bits, or about 25.33% of that fixed-length payload.

This comparison excludes the extra information needed to communicate the codebook or tree, along with any framing or padding. For a very short message, that overhead can cancel the payload savings. Huffman coding does not promise that every file becomes smaller.

9.6 A powerful cross-check: sum the merge weights

The same total is

$$14 + 25 + 30 + 55 + 100 = 224.$$

Why? Each merge moves every symbol in its two groups one edge farther from the eventual root. That adds one bit for every occurrence of every symbol in those groups, exactly the combined weight. Summing all merge weights counts the full weighted leaf depth.

9.7 Encoding and decoding

To encode, replace each symbol by its code and join the bits. In the example,

$$\text{FACE} = 0 \mid 1100 \mid 100 \mid 111 = 01100100111.$$

The vertical separators are shown only for explanation; a prefix-free stream does not need them between symbols.

To decode, start at the root. A 0 means move left; a 1 means move right. When a leaf is reached, output that symbol and return to the root before reading the next bit. For the stream above, the successive leaf paths are 0, 1100, 100, and 111, giving FACE. If the stream ends at an internal node, it contains an incomplete final codeword unless some external framing rule says otherwise.

The decoder must know the same code tree or equivalent codebook used by the encoder. Frequencies alone can be insufficient to reproduce exact bit labels unless the tie-breaking and edge-label conventions are also shared.

Algorithm: Encode a message with a known Huffman codebook

- Step 1. Start.** Receive the message and its matching codebook. Begin with an empty output bit stream.
- Step 2.** If no message symbols remain, go to Step 6.
- Step 3.** Read the next symbol.
- Step 4.** Look up its complete code. If the symbol is absent from the codebook, report that the message cannot be encoded with this codebook and end.
- Step 5.** Append that code to the output stream and repeat from Step 2.
- Step 6. End.** Return the bit stream together with whatever codebook and framing information the chosen format requires.

Algorithm: Decode a Huffman bit stream

Input: a valid matching tree and a bit stream. **Output:** the decoded symbols, or an invalid-stream report.

- Step 1. Start.** Receive the tree and bit stream; begin with an empty output message.

Step 2. If the tree is empty, accept only an empty stream and end. If there is only one leaf and the chosen code is 0, output that symbol for each 0; reject any 1; end. Otherwise continue with the multi-leaf tree.

Step 3. Set the current node to the root.

Step 4. If there are no unread bits, go to Step 9.

Step 5. Read the next bit. For 0 move to the left child; for 1 move to the right child. Reject anything other than a bit, or any move to a missing child.

Step 6. If the new current node is internal, repeat from Step 4.

Step 7. If it is a leaf, append that leaf's symbol to the output.

Step 8. Reset the current node to the root and repeat from Step 4.

Step 9. If the current node is not the root, report an incomplete final codeword. Otherwise accept the output message.

Step 10. End. Return the output message or the error report.

Implementation focus. The essential operations are minimum selection, creation of a parent, and traversal of its children. Their numbered algorithms are shown in full. A separate full C priority-queue implementation for Huffman coding would repeat much of the heap machinery, so this section concentrates on the tree, codes, and bit-by-bit trace.

9.8 Why combining the smallest weights makes sense

Deep leaves pay for many bits each time their symbols appear. Giving long paths to infrequent symbols hurts less than giving them to frequent symbols. In an optimal binary prefix tree, two smallest-weight symbols can be placed as deepest sibling leaves. Combining them into one group reduces the problem to a smaller problem of the same kind.

This is a **greedy** method: make a locally cheapest choice, then continue. Greedy choices are not automatically correct for every problem. Huffman's method is correct because the sibling-and-combination argument shows that its first choice can fit into an optimal solution, and the remaining smaller problem can be solved in the same way.

The guarantee is minimum weighted code length among binary prefix codes for the given symbol weights. It is not a claim that Huffman coding beats every possible compression method on every kind of data.

9.9 Ties, edge cases, and non-uniqueness

- Equal weights can be combined in different valid orders. Left and right children can also be swapped. Exact codes can therefore differ while total cost stays optimal.
- More frequent symbols do not necessarily receive *strictly* shorter codes. Several frequencies can share one code length, as C, D, and E do here.
- A single distinct symbol creates a one-leaf tree. Mathematically its root-to-leaf path has length 0. Practical formats often assign it one bit per occurrence, or store a count alongside an empty code. State the convention used.
- No input symbols means there is no nonempty Huffman tree to build.

- Zero-frequency symbols are normally omitted when building a code for the observed message. If a format requires them, that is an additional condition.
- With $k \geq 1$ leaves in the ordinary full Huffman tree, there are $k - 1$ internal nodes and $2k - 1$ total nodes.

9.10 Huffman analysis

Counting frequencies in an input of N symbols takes $O(N)$ time under constant-time counting access. Build a min-heap of the k distinct positive-frequency symbols in $O(k)$ time. Each of $k - 1$ rounds performs two removals and one insertion, each costing $O(\log k)$. Tree construction is therefore $O(k \log k)$, and counting plus construction is $O(N + k \log k)$.

The tree, frequencies, and heap use $O(k)$ working storage. If all codewords are explicitly materialized as separate bit strings, their total stored length is an additional cost and can reach $O(k^2)$ in a highly uneven code tree. An ordinary traversal touches the $O(k)$ tree nodes, but writing out all codeword bits is proportional to how many bits are written.

Encoding also takes time to write the B output bits, so a suitable bound is $O(N + B)$ once codes are available. Tree-based decoding reads the bit stream one bit at a time and takes $O(B)$ time for a nondegenerate code, with output storage counted separately. If weights are already sorted, a specialized two-queue construction can build the tree in $O(k)$ time; the standard min-heap method is the main method here.

10 Analysis of algorithms in simple English

10.1 What are we measuring?

An **algorithm** is a precise sequence of steps for completing a task. Analysis estimates how its work and memory requirements grow when the input gets larger. It usually does not try to predict an exact number of seconds, because that depends on the machine, representation, and surrounding conditions.

First name the input size. For tree search it is often the number of nodes n , but the actual route length depends on height h . For Huffman coding there are two important sizes: input length N and distinct-symbol count k . Mixing them hides important costs.

Then decide what one unit of work means. Comparing two ordinary keys, following a child link, and exchanging two fixed-size values are often treated as constant-time actions. If values are extremely long strings or large objects, their comparison or movement can have its own cost. The standard bounds in these notes assume ordinary constant-time key operations.

10.2 Growth rates you should recognize

Growth	Plain-English meaning	Example in this unit
1	Work does not grow with n	Read a heap root
$\log n$	Only a few extra steps when size multiplies	Search a balanced tree
n	Work roughly follows number of items	Visit every node
$n \log n$	About logarithmic work per item overall	Heap sort worst case
n^2	Work can grow like all pairs or a long sum	Build a chain by repeated BST insertion

For $n = 1024$, these expressions have values 1, 10, 1024, 10240, and 1048576 when the logarithm has base 2. These are illustrative growth values, not measured instruction counts.

10.3 What a logarithm means here

$\log_2 n$ asks how many times you can approximately halve n before reaching 1, or how many doublings take 1 to n . Since $2^{10} = 1024$, $\log_2 1024 = 10$. Doubling a large balanced tree adds only about one level.

A complete tree with 1024 nodes has height 10, which means a root-to-deepest-node path visits 11 nodes. This is why exact counts and growth notation are different: h counts edges, while a search can inspect $h + 1$ nodes.

The base of a logarithm changes it only by a constant factor, so $O(\log_2 n)$ and $O(\log_{10} n)$ describe the same broad growth class. Exact height calculations still require the correct base, usually 2 for binary trees.

10.4 Big O, Omega, and Theta

Big O, written $O(f(n))$, gives an upper growth bound: beyond some input size, the work is no more than a constant times $f(n)$.

Omega, written $\Omega(f(n))$, gives a lower growth bound: beyond some input size, the work is at least a positive constant times $f(n)$.

Theta, written $\Theta(f(n))$, gives both: the work grows at the same broad rate as $f(n)$, up to constant factors.

If a traversal always visits n nodes and does bounded work at each node, its cost is $\Theta(n)$. Calling it $O(n^2)$ is a mathematically valid upper bound, but a much less informative one.

Common mistake. Big O does not itself mean “worst case”, and Omega does not itself mean “best case”. You can give an upper or lower bound for a best-case, average-case, or worst-case cost function. Always identify the case.

10.5 Best, worst, and average cases

For searching a fixed nonempty BST:

- **Best case:** the root is the desired key, requiring constant work.
- **Worst case:** follow a longest relevant path, using $O(h + 1)$ work.
- **Average or expected case:** depends on how trees and queries are chosen. A probability assumption is needed.

An ordinary BST can be good on average under random insertion order and still be poor for sorted input. An AVL tree controls the shape, so its logarithmic worst-case search does not depend on a fortunate insertion order.

10.6 Why lower-order terms are dropped

Suppose a process takes $3n^2 + 7n + 20$ elementary steps. For large n , the square term dominates, so its growth is $\Theta(n^2)$. This does not mean the smaller terms are literally zero. It means they do not change the long-run growth class.

Two sequential stages add their costs. Building a heap and then sorting costs

$$\Theta(n) + O(n \log n) = O(n \log n).$$

Repeated work must be counted carefully. Running a logarithmic operation n times often gives $O(n \log n)$, but bottom-up heap construction is faster because different nodes have different amounts of possible downward work.

10.7 Useful sums and recurrences

A **recurrence** is an equation describing a problem’s work in terms of smaller versions of the same problem.

Balanced one-path search. If each comparison leaves about half the candidates,

$$T(n) = T(n/2) + O(1), \quad T(n) = O(\log n).$$

This model assumes the tree is balanced enough for the remaining subtree to shrink in that way. It is not the recurrence for a chain.

Chain search. If each step removes only one candidate,

$$T(n) = T(n-1) + O(1), \quad T(n) = O(n).$$

Full traversal. If left and right subtree sizes are n_L and n_R ,

$$T(n) = T(n_L) + T(n_R) + O(1), \quad n_L + n_R = n - 1.$$

Every node contributes once, so total time is $\Theta(n)$ for balanced and skewed trees alike.

Repeated increasing-length work.

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2).$$

This explains the worst case for repeated ordinary BST insertion and for reconstruction with repeated long inorder scans.

10.8 Space: stored data versus temporary help

Total space includes the input representation, temporary storage, and sometimes the output. **Auxiliary space** usually means extra temporary storage beyond the input; whether output is included should be stated.

An n -node tree takes $\Theta(n)$ storage. Searching it iteratively can still have $O(1)$ auxiliary space. A recursive traversal can need $O(h+1)$ temporary reminder space while also creating an n -item output list. Those are different costs and should not be combined without explanation.

An algorithm that rearranges an existing array using only a constant number of extra storage slots is called **in-place** under the strict auxiliary-space convention. Standard iterative heap sort fits this description.

10.9 How to write a good analysis answer

1. State the size variables and any important assumptions.
2. Explain what path, nodes, levels, or rounds are processed.
3. Count how many times the main action happens.
4. Give a bound and name the case: best, expected, or worst.
5. State auxiliary space separately from input and output storage.

For example: “AVL insertion follows a path of height $O(\log n)$ and performs constant work per visited ancestor, plus at most two local rotations. Its worst-case time is $O(\log n)$. A recursive approach uses $O(\log n)$ extra space; the tree itself uses $\Theta(n)$ space.” This explains the result rather than merely listing it.

11 Connections, applications, and revision sheets

11.1 Which structure solves which problem?

Structure	Main rule	What it makes convenient
Binary tree	At most two child positions	Represent branching relationships or expressions
BST	All left keys smaller; all right keys larger	Search by key, ordered traversal, predecessor and successor
AVL tree	BST rule plus height difference at most 1 everywhere	Guaranteed short search and update paths
Binary heap	Complete shape plus parent-child priority	Repeatedly retrieve and remove the extreme-priority item
Huffman tree	Symbols at leaves; formed by combining least weights	Construct an optimal binary prefix code for symbol weights

An AVL tree is a BST, and both are binary trees. A binary heap is also a binary tree, but generally not a BST. A Huffman tree is full when built by pairwise merging positive-weight leaves, but it need not be complete, AVL-balanced, or a search tree.

11.2 Applications explained through their main operation

Folders and organization charts: the key operation is following parent-child relationships. These often use general trees because a folder or manager can have more than two children.

Expression processing: operators combine child results. Postorder is natural when the child values must be computed before the operator.

A collection that needs sorted output and updates: a balanced search tree can add and remove keys while supporting inorder output. To report keys in a range, a suitable traversal can skip subtrees entirely outside the allowed range. With balanced height, reporting r matching keys can take $O(\log n + r)$ time.

A job scheduler: the main operation may be “give me the highest-priority waiting job.” A heap is natural. If jobs with equal priority must follow arrival order, store arrival order as a secondary comparison key; the heap does not guarantee this automatically.

Keeping the largest r items seen so far: use a min-heap of at most r candidates. Its root is the weakest retained candidate. A larger incoming value can replace that root. For $r \geq 2$, processing n values costs $O(n \log r)$ time and $O(r)$ candidate storage; for $r = 1$, simply track a maximum.

Data compression: Huffman coding uses a tree to turn frequent symbols into short leaf paths. A min-heap supports building that tree, illustrating how one data structure can help construct another.

11.3 Operation-cost reference

Unless noted otherwise, time entries are worst-case upper bounds for nonempty structures; ordinary constant-time key operations are assumed.

Operation	Time	Extra space / qualification
Ordinary BST search, insert, delete	$O(h+1)$; up to $O(n)$	Path reminders if recursive
AVL search and insertion	$O(\log n)$	$O(\log n)$ if recursive
One rotation	$O(1)$	$O(1)$; local heights available
Any complete traversal	$\Theta(n)$	DFS: $O(h+1)$; BFS: $O(w)$
Preorder/inorder or postorder/inorder reconstruction with lookup	$O(n)$	$O(n)$ lookup plus $O(h+1)$ reminders; output separate
Reconstruction with repeated scans	$O(n^2)$	Shape and copying choices affect details
Read heap root	$\Theta(1)$	Nonempty heap
Heap insert / remove root / sift down	$O(\log n)$	$O(1)$ with iterative repair
Bottom-up heap construction	$\Theta(n)$	$O(1)$ with array and iterative repair
Heap sort	$\Theta(n \log n)$ worst case	$O(1)$ with iterative array method
Huffman tree from k weights using min-heap	$O(k \log k)$	$O(k)$ for heap and tree
Huffman frequency counting plus construction	$O(N + k \log k)$	Encoded output handled separately

11.4 Compact formula sheet

Edges in a nonempty tree : $n - 1$.

Maximum binary-tree nodes at depth d : 2^d .

Node bounds for height h : $h + 1 \leq n \leq 2^{h+1} - 1$.

Complete binary-tree height : $\lfloor \log_2 n \rfloor$.

Leaves and two-child nodes : $n_0 = n_2 + 1$.

Empty binary child links : $n + 1$.

One-based left and right children : $2i, 2i + 1$.

One-based parent : $\lfloor i/2 \rfloor$ ($i > 1$).

Zero-based left and right children : $2i + 1, 2i + 2$.

Zero-based parent : $\lfloor (i - 1)/2 \rfloor$ ($i > 0$).

AVL balance factor : $h_L - h_R$.

Huffman payload cost : $\sum_s f_s \ell_s$.

In the last formula, f_s is a symbol's frequency and ℓ_s is its code length. The sum is taken over all represented symbols.

11.5 Frequent exam mistakes and their corrections

1. **Counting nodes when height uses edges.** A one-node tree has height 0 here.
2. **Calling every binary tree a BST.** A binary tree has a child-count rule; a BST adds an ordering rule.

3. **Checking only immediate children in a BST.** A descendant must obey all ancestor bounds.
4. **Assuming inorder is always sorted.** It is sorted for a BST, not for every binary tree.
5. **Reconstructing a general tree from preorder and postorder as though it were unique.** A single child can be on either side.
6. **Deleting a two-child BST node without repairing the replacement's old position.** Remove the old successor or predecessor occurrence too.
7. **Using an LL rotation for an LR bend.** An inside bend needs two rotations.
8. **Balancing only the root.** The AVL rule holds at every node.
9. **Building a heap top-down with one sift down per node.** A sift-down call assumes its child subtrees already satisfy the heap rule; bottom-up order supplies that condition.
10. **Treating a heap array as sorted.** Only parent-child priority is guaranteed.
11. **Including the sorted suffix in heap-sort repair.** Repair only the active heap.
12. **Selecting the two smallest original Huffman symbols every round.** Combined groups participate in later selections.
13. **Comparing exact Huffman codes when ties exist.** Check valid construction, prefix freedom, and cost; multiple codebooks can be correct.
14. **Giving a bound without assumptions.** Say which operation, case, tree shape, and storage convention you mean.

11.6 A suggested learning sequence

1. Learn the tree vocabulary and draw the four binary-tree shapes.
2. Practise all four traversals on one tree until you can explain each order.
3. Reconstruct trees using inorder as the left/right divider.
4. Draw BST searches, insertions, and all three deletion cases.
5. Practise the four AVL rotation cases, then mixed insertion sequences.
6. Keep a heap's tree drawing beside its array while tracing repairs and sorting.
7. Build a Huffman tree, assign bits, encode, decode, and verify the cost twice.
8. Explain operation costs using paths, node visits, levels, or merges.

11.7 How to connect an algorithm, a picture, and a C snippet

Use this three-pass method on each operation. First draw the input and predict the output. Next trace the numbered algorithm, writing the current node or active array boundary beside each repeated step. Finally read the matching C function and identify where those same decisions occur.

Operation	What to track on paper	What to notice in C
Traversal	Current node, paused calls, visit order	Position of the print relative to the two child calls
BST insertion	Comparisons and the first empty child position	Returned subtree root assigned to the selected child
BST deletion	Child count and successor's original position	Save surviving child before freeing; attach returned subtree
AVL insertion	Heights while returning upward; first imbalance	Update height, identify outside or inside bend, keep rotated root
Heap repair	Current index and larger child	Child bounds checked inside the active item count
Heap sort	Active region and sorted suffix	Repair receives a shorter item count each round
Huffman coding	Two lowest weights, current path bits	The same child-reference and traversal ideas; no extra listing needed

Small C reading guide. A function parameter such as `Node *root` holds a reference, not the whole tree. Reassigning that local parameter changes the function's current reference. Assigning `root->left` changes a connection stored in an actual node. These two actions explain why search leaves the structure untouched but insertion and rotation can change it.

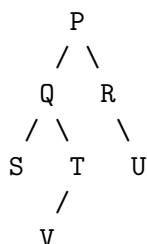
The functions are intended as one shared teaching set: use the headers and `Node` definition once, then the required functions in the order shown. The snippets use C99 or later, where declarations can appear after earlier statements and inside a loop header. Tests and a menu-driven program are deliberately kept out of the notes so the implementation remains a small part of the guide.

12 Practice problems: draw, reason, calculate

Try these without looking at the answer section. For every tree operation, show the affected path and the final tree. Use the conventions stated at the beginning. Problems marked **Stretch** require a short argument rather than a long calculation.

12.1 Foundations and counting: Problems 1–6

Use this tree for Problems 1 and 2:



Problem 1. Identify the root, leaves, internal nodes, siblings of T, parent of V, and ancestors of V.

Problem 2. Find the depth of V, height of T, height of Q, height of the tree, degree of R, and number of edges.

Problem 3. A nonempty binary tree has height 4. What are its smallest and largest possible node counts? How many occupied levels does it have?

Problem 4. A full binary tree has 12 internal nodes. Find its leaves, total nodes, and edges. Could a full binary tree have exactly 20 total nodes? Explain.

Problem 5. Classify each shape as full, complete, perfect, or none of those: (a) root with two leaf children; (b) root with only a left leaf child; (c) root with only a right leaf child; (d) root with two children, whose right child has two leaf children and whose left child is a leaf. List all applicable classes.

Problem 6. Stretch. A binary tree has 8 leaves and 5 nodes with exactly one child. How many nodes have two children, and how many total nodes are there? Explain your counting rule.

12.2 Memory representation: Problems 7–10

Problem 7. In one-based positional storage, a node is at position 5. Give its parent position and both child positions. If this is a complete tree with 10 nodes, which children actually exist?

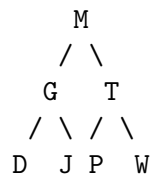
Problem 8. In zero-based positional storage, a node is at index 4. Give the parent and child indices. If the array has 10 entries, which child indices exist?

Problem 9. A linked binary tree has 15 nodes and two child-link slots per node. Count the nonempty child links and empty child links, excluding the separate root reference.

Problem 10. A five-node tree is a right-only chain. Give its one-based positional indices. Compare the required positional-array span with the number of linked records.

12.3 Traversals: Problems 11–16

Use this tree for Problems 11–14:



Problem 11. Write preorder, inorder, postorder, and level order.

Problem 12. During level order, show the waiting line after visiting M, then G, then T. Put left children before right children.

Problem 13. Why does inorder produce alphabetically sorted labels for this particular tree? Would any binary tree with these labels do so?

Problem 14. If visiting each node takes one unit of work, how many visits does a complete postorder traversal make? What is its time class? What is the tree's height?

Problem 15. Draw an expression tree for $(A - B) \times (C + D)$. Write its preorder and postorder sequences. Evaluate it for $A = 8, B = 3, C = 2, D = 4$.

Problem 16. A tree is a 100-node chain. Compare depth-first recursive reminder space and level-order queue space. Both procedures visit all nodes.

12.4 Construction from traversals: Problems 17–22

Problem 17. Construct the tree with preorder M, G, D, J, T, P, W and inorder D, G, J, M, P, T, W. Show the first split.

Problem 18. Construct the tree with postorder D, J, G, P, W, T, M and inorder D, G, J, M, P, T, W. Identify the root of each main subtree.

Problem 19. Construct the tree with preorder A, B, D, E, G, C, F and inorder D, B, G, E, A, F, C. Then write its postorder.

Problem 20. Give two different binary trees with preorder A, B, C and postorder C, B, A. What information is missing?

Problem 21. Can preorder A, B, C and inorder C, A, B describe the same binary tree? Show the first contradiction.

Problem 22. Stretch. Explain why repeated inorder scanning can make construction quadratic. What two changes allow linear-time construction under the usual constant-time lookup assumption?

12.5 Binary search trees: Problems 23–29

Problem 23. Insert 40, 20, 60, 10, 30, 50, 70 into an empty ordinary BST. Draw the final tree and give its inorder.

Problem 24. Using the tree from Problem 23, list the comparisons when searching for 50 and for 55. State how the unsuccessful search ends.

Problem 25. Starting from Problem 23's tree, delete 10, then 20. Draw the final result and name the deletion case used at each step.

Problem 26. Using a fresh copy of Problem 23's tree, delete 40 using its inorder successor. Draw the result and write its inorder.

Problem 27. A tree has root 40, left child 20, right child 60, and 20 has right child 45. Is it a valid BST? Identify the violated bound.

Problem 28. Insert 5, 10, 15, 20, 25 into an empty ordinary BST. Find the height and the number of node comparisons when searching for 25. Explain the worst-case cost of building an n -node tree with increasing input.

Problem 29. In Problem 23's tree, find the predecessor and successor of 40, the successor of 30, and the predecessor of 10. Explain the cases where the answer is an ancestor or does not exist.

12.6 AVL trees: Problems 30–36

Problem 30. A node has left-subtree height 2 and right-subtree height 0. Find its balance factor and height. Is it AVL-balanced?

Problem 31. For each fresh empty AVL tree, insert the three keys and name the case and repair: (a) 30, 20, 10; (b) 10, 20, 30; (c) 30, 10, 20; (d) 10, 30, 20. Draw the final trees.

Problem 32. Insert 40, 20, 60, 10, 30, 25 into an empty AVL tree. Identify the first unbalanced ancestor after the last insertion, show the two rotations, and draw the final tree.

Problem 33. In a right rotation, z is 50, its left child y is 30, and y 's right subtree is rooted at 40. Where must the subtree rooted at 40 go? Explain using key ranges.

Problem 34. A node has one leaf child on the left and no right child. Find its height and balance factor using this guide's convention. Does it need a rotation?

Problem 35. Stretch. Use the minimum-node recurrence to calculate the smallest number of nodes possible in an AVL tree of height 4. Show the counts for heights 0 through 4.

Problem 36. Explain why “the root has balance factor 0” does not prove that the entire tree is AVL. State the two properties that must hold throughout the tree.

12.7 Heaps and heapify: Problems 37–44

Problem 37. Is [90, 70, 80, 20, 60, 50] a valid max-heap? Is it a BST when drawn as a complete tree? Justify both answers.

Problem 38. Is [10, 20, 15, 30, 5] a valid min-heap? If not, identify an exact offending parent-child pair.

Problem 39. Build a max-heap bottom-up from [3, 1, 6, 5, 2, 4]. Show the array after processing each internal position.

Problem 40. Insert 8 into the max-heap [6, 5, 4, 1, 2, 3]. Show every exchange.

Problem 41. Remove the maximum from [8, 5, 6, 1, 2, 3, 4]. Show root replacement, repair, and the returned value.

Problem 42. A complete binary heap has 11 nodes. Find its height, last internal position, and leaf positions using one-based numbering.

Problem 43. Explain why max-heap sift down must choose the larger child, using parent 8 with children 12 and 20.

Problem 44. Stretch. Explain why bottom-up build-heap is $\Theta(n)$ even though a single repair may take $O(\log n)$. Contrast it with repeated insertion.

12.8 Heap sort: Problems 45–48

Problem 45. Sort [3, 1, 4, 2] in ascending order with in-place heap sort. Show the initial max-heap and each array after an extraction and repair. Separate the active heap from the sorted suffix.

Problem 46. Which heap type is used for ascending in-place heap sort? Which heap type is used if you repeatedly remove the root and append it to a separate list to obtain ascending order? Explain the difference.

Problem 47. Why must heap-sort repair ignore the sorted suffix? State a property of the active region and a property of the suffix that hold after every complete round.

Problem 48. Explain heap sort's worst-case time, auxiliary space with iterative repair, and stability. Give a reason for each answer.

12.9 Huffman coding: Problems 49–54

Problem 49. Build a Huffman tree for A:5, B:7, C:10, D:15. List all merges, provide one valid codebook, and find total payload bits and average bits per symbol.

Problem 50. Using Problem 49, compare the Huffman payload with a fixed-length code for the same four symbols. State the number of bits saved and the percentage saved, excluding codebook overhead.

Problem 51. A codebook is A=0, B=10, C=110, D=111. Is it prefix-free? Encode BAD and decode 011010111.

Problem 52. A proposed codebook is A=0, B=01, C=1. Show two decodings of 01 and explain the prefix problem.

Problem 53. Build a Huffman tree for four symbols of equal frequency 1. What are the code lengths and total payload cost? Must two correct solutions have identical bit strings?

Problem 54. Stretch. For weights 2, 3, 7, 9, list the Huffman merge weights and use their sum to obtain the encoded cost. Explain why this agrees with frequency times leaf depth.

12.10 Analysis and mixed reasoning: Problems 55–60

Problem 55. A complete binary tree has 1023 nodes. Find its height, number of levels, largest possible number of node comparisons along one search path, and visits in a complete traversal.

Problem 56. Compare ordinary BST search, AVL search, and arbitrary-value heap search in the worst case. Explain why logarithmic heap height does not guarantee logarithmic arbitrary-value search.

Problem 57. A process performs bottom-up heap construction on n values, then removes the root n times. Give a total worst-case time bound and explain why the linear build step does not dominate.

Problem 58. You must repeatedly take the earliest event, and each event has a time. Would a min-heap or max-heap be more natural? If you also need sorted range queries by event time, what alternative tree becomes useful?

Problem 59. A Huffman input has N symbols in total and k distinct symbols. State the frequency-counting cost, min-heap tree-construction cost, and combined cost. Why are N and k different?

Problem 60. Stretch. Correct each claim: (a) every balanced tree traversal is logarithmic; (b) every BST is AVL; (c) every max-heap is descending as an array; (d) preorder and postorder always uniquely determine a binary tree; (e) Big O automatically describes the worst case.

13 Answers and worked solutions

These solutions use the stated conventions. For Huffman codes, other bit assignments can be equally correct. For BST deletion, predecessor replacement can produce a different correct tree when the question does not specify the successor.

13.1 Solutions 1–10: foundations and memory

Answer 1. Root: P. Leaves: S, V, U. Internal nodes: P, Q, R, T. T's sibling is S. V's parent is T. V's ancestors are T, Q, P. A node is not counted as its own ancestor under our convention.

Answer 2. V has depth 3 along P–Q–T–V. T has height 1, Q has height 2, and the whole tree has height 3. R has one child, so its rooted-tree degree is 1. There are 7 nodes and $7 - 1 = 6$ edges.

Answer 3. Minimum nodes: $h+1 = 5$, achieved by a chain. Maximum nodes: $2^{h+1} - 1 = 2^5 - 1 = 31$, achieved by a perfect tree. There are $h + 1 = 5$ occupied levels.

Answer 4. There are $12 + 1 = 13$ leaves, $12 + 13 = 25$ total nodes, and 24 edges. A nonempty full binary tree has $2I + 1$ total nodes for some internal-node count I , so its total is odd. Exactly 20 total nodes is impossible.

Answer 5. (a) Full, complete, and perfect. (b) Complete only: the final level fills its leftmost position, but the root has exactly one child. (c) None: it is neither full nor perfect, and a right child without a left child leaves a gap in the last level. (d) Full only: every internal node has two children, but the last-level positions under the left subtree are empty while positions farther right are occupied; leaf depths also differ.

Answer 6. Every nonempty binary tree satisfies $n_0 = n_2 + 1$. Therefore $n_2 = 8 - 1 = 7$. Total nodes are $8 + 5 + 7 = 20$. The edge check gives $5 + 2(7) = 19 = 20 - 1$. Nodes with one child do not change the leaves-minus-two-child-nodes difference.

Answer 7. Parent position is $\lfloor 5/2 \rfloor = 2$. Children are at 10 and 11. In a complete tree containing positions 1 through 10, only the left child at 10 exists.

Answer 8. Parent index is $\lfloor (4 - 1)/2 \rfloor = 1$. Children are at $2(4) + 1 = 9$ and $2(4) + 2 = 10$. A 10-entry zero-based array occupies indices 0 through 9, so only index 9 exists.

Answer 9. There are $15 - 1 = 14$ real child connections. The 15 nodes provide 30 child-link slots, leaving $30 - 14 = 16$ empty child links. This matches $n + 1$.

Answer 10. Positions are 1, 3, 7, 15, 31. The positional scheme needs a span through position 31, of which only five positions hold nodes. Linked storage needs five node records, each with its ordinary fields. The array's empty positions are the issue, even though both represent the same five-node tree.

13.2 Solutions 11–16: traversals

Answer 11. The four orders are:

- Preorder: M, G, D, J, T, P, W.
- Inorder: D, G, J, M, P, T, W.

- Postorder: D, J, G, P, W, T, M.
- Level order: M, G, T, D, J, P, W.

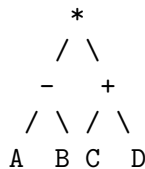
Check that preorder starts at M, postorder ends at M, and every sequence contains all seven labels once.

Answer 12. After M, the queue is G, T. After G, it is T, D, J. After T, it is D, J, P, W. Children join the back while the oldest waiting node leaves the front.

Answer 13. The tree satisfies the BST ordering rule when alphabetical order is used to compare labels. Inorder finishes all smaller left-subtree labels, visits the current label, then finishes all larger right-subtree labels. An arbitrary binary tree with the same labels need not satisfy that ordering.

Answer 14. There are exactly 7 visits. The time class is $\Theta(n)$ for n nodes because each is visited once. The pictured tree has height 2 and three occupied levels. Height does not reduce the number of nodes a complete traversal must visit.

Answer 15. The root is multiplication, with subtraction on the left and addition on the right.

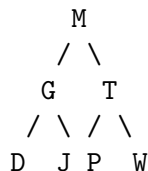


Preorder is $*, -, A, B, +, C, D$. Postorder is $A, B, -, C, D, +, *$. The left subtree gives $8 - 3 = 5$; the right gives $2 + 4 = 6$; the result is $5 \times 6 = 30$.

Answer 16. An ordinary recursive depth-first traversal can keep reminders for all 100 nodes along the chain, so its auxiliary space is $O(n)$. Level order needs only one waiting node at a time because each node has at most one child, so queue space is $O(1)$. Both take $\Theta(n)$ time. If their outputs are stored, both additionally require $\Theta(n)$ output space.

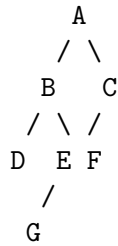
13.3 Solutions 17–22: reconstruction

Answer 17. M is first in preorder, so it is the root. Inorder splits into (D, G, J) , M, (P, T, W) . Three preorder items belong to each subtree. G roots the left subtree with children D and J; T roots the right subtree with children P and W.



Answer 18. M is last in postorder, so it is the root. The inorder groups have three nodes each. Their postorder segments are D, J, G and P, W, T. Thus G and T are the respective subtree roots. Splitting each inorder segment gives the same final tree as Answer 17.

Answer 19. Root A splits inorder into left group D, B, G, E and right group F, C. The left preorder group is B, D, E, G; the right is C, F. Within the left group, B has left child D and right subtree rooted at E; G lies to E's left. Within the right group, C has left child F.



Postorder is D, G, E, B, F, C, A. Recomputing the given preorder and inorder from this picture is a useful final check.

Answer 20. An all-left chain A–B–C and an all-right chain A–B–C both have preorder A, B, C and postorder C, B, A. The traversals identify the ancestor order but not the side of a lone child. Mixed-side chains are additional possibilities.

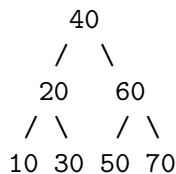


Answer 21. No. A must be the root. Inorder puts C in its left subtree and B in its right subtree. Preorder must visit the left subtree immediately after A, but the next item is B. That contradicts the required left-subtree label C.

Answer 22. On an unfavourable shape, scans can examine segments of length $n, n-1, \dots, 1$, whose sum is $\Theta(n^2)$. Prepare a label-to-inorder-position lookup, and use index boundaries to identify segments without repeatedly copying them. With constant-time lookup and constant bookkeeping per node, total construction time becomes $O(n)$.

13.4 Solutions 23–29: BST operations

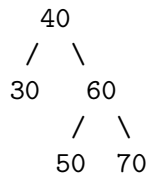
Answer 23. 40 becomes the root, 20 and 60 become its children, and the remaining keys fill the appropriate leaf positions.



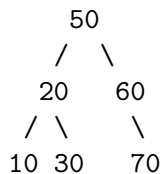
Inorder is 10, 20, 30, 40, 50, 60, 70.

Answer 24. For 50, compare with 40, move right to 60, then left to 50 and succeed. For 55, compare with 40, 60, and 50, then try 50's right child. It is empty, so the search fails. Both inspect three stored keys, but the unsuccessful search then encounters an empty position.

Answer 25. 10 is a leaf, so its deletion is case 1. Then 20 has only one child, 30, so deleting 20 is case 2: connect 40 directly to 30.



Answer 26. The successor of 40 is 50, the minimum of its right subtree. Replace 40 by 50, then remove the old leaf 50 below 60.



Inorder is 10, 20, 30, 50, 60, 70. It contains exactly the original keys except 40.

Answer 27. It is invalid. The key 45 lies in 40's left subtree, which imposes the upper bound $45 < 40$. Although 45 is correctly larger than its immediate parent 20, it violates the ancestor's bound. Its allowed range at that position is strictly between 20 and 40.

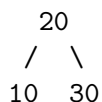
Answer 28. The result is a five-node right chain, with height 4. Searching for 25 compares with 5, 10, 15, 20, and 25: five node comparisons. Building an n -node increasing chain performs about $0 + 1 + \dots + (n - 1)$ comparisons, giving $\Theta(n^2)$ total time.

Answer 29. Predecessor of 40: 30, the maximum of its left subtree. Successor of 40: 50, the minimum of its right subtree. Successor of 30: 40; 30 has no right subtree, and 40 is the first ancestor reached whose left side contains 30. Predecessor of 10: none, because 10 is the minimum key in the tree.

13.5 Solutions 30–36: AVL trees

Answer 30. Balance factor is $2 - 0 = +2$, so the node is not AVL-balanced. Its own height is $1 + \max(2, 0) = 3$. The positive sign tells us the left subtree is taller, but the child configuration is still needed to choose the repair.

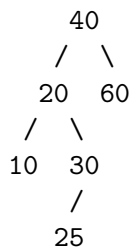
Answer 31. (a) LL, right rotation at 30. (b) RR, left rotation at 10. (c) LR, left rotation at 10 followed by right rotation at 30. (d) RL, right rotation at 30 followed by left rotation at 10. All four finish with the same three-key tree:



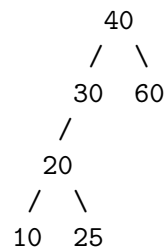
All nodes are balanced and inorder remains 10, 20, 30.

Answer 32. Before inserting 25, root 40 has left subtree rooted at 20 with children 10 and 30, and right leaf 60. The new 25 becomes 30's left child. Node 30 remains balanced, node 20 has BF -1 , but node 40 has BF $+2$. The first unbalanced ancestor is therefore 40, with an LR path.

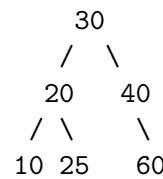
Before repair:



Left at 20:



Right at 40:



The final inorder is 10, 20, 25, 30, 40, 60. Nodes 20 and 40 have height 1; root 30 has height 2. The BST and AVL rules both hold.

Answer 33. The subtree rooted at 40 becomes the left subtree of 50. Its keys lie between 30 and 50: they must stay to the right of 30 but to the left of 50. After rotation, 30 is above 50, so attaching that middle subtree on 50's left preserves both bounds.

Answer 34. The left leaf has height 0 and the empty right subtree has height -1 . The node's height is $1 + \max(0, -1) = 1$, and its balance factor is $0 - (-1) = +1$. It is balanced and needs no rotation.

Answer 35. With $M(-1) = 0$ and $M(0) = 1$, compute $M(1) = 1 + 1 + 0 = 2$, $M(2) = 1 + 2 + 1 = 4$, $M(3) = 1 + 4 + 2 = 7$, and $M(4) = 1 + 7 + 4 = 12$. The minimum for height 4 is 12 nodes. The growth comes from needing two reasonably tall child subtrees at every tall node.

Answer 36. A root can have equally tall children while one of those children's own subtrees differs in height by 2 or more. The root's factor checks only the root. An AVL tree must satisfy the BST ordering rule throughout and the balance-factor condition $-1, 0, +1$ at every node.

13.6 Solutions 37–44: heaps

Answer 37. It is a valid max-heap: $90 \geq 70, 80$; $70 \geq 20, 60$; and $80 \geq 50$. The consecutive array positions give a complete shape. It is not a BST because 80 lies in the root's right subtree but is smaller than 90.

Answer 38. It is not a min-heap. Position 2 contains 20 and its right child at position 5 contains 5, violating $20 \leq 5$. The root being the smallest of its immediate children is insufficient; every parent-child relationship must be valid.

Answer 39. The last internal position is 3. Process positions 3, 2, 1:

- At 3, value 6 has child 4; no exchange. Array: $[3, 1, 6, 5, 2, 4]$.
- At 2, value 1 has children 5 and 2; exchange with 5. Array: $[3, 5, 6, 1, 2, 4]$.
- At 1, exchange 3 with larger child 6: $[6, 5, 3, 1, 2, 4]$. Continue at position 3 and exchange 3 with 4: $[6, 5, 4, 1, 2, 3]$.

The final array is a valid max-heap.

Answer 40. Append 8 to get $[6, 5, 4, 1, 2, 3, 8]$. Exchange 8 with its parent 4 at position 3: $[6, 5, 8, 1, 2, 3, 4]$. Exchange 8 with its parent 6 at position 1: $[8, 5, 6, 1, 2, 3, 4]$. It has reached the root.

Answer 41. Save and return 8. Replace the root with the last value 4 and shorten the array: $[4, 5, 6, 1, 2, 3]$. Exchange 4 with larger child 6: $[6, 5, 4, 1, 2, 3]$. The value 4 now has child 3, so the heap rule holds and repair stops.

Answer 42. Height is $\lfloor \log_2 11 \rfloor = 3$. The last internal position is $\lfloor 11/2 \rfloor = 5$. Leaves occupy positions 6, 7, 8, 9, 10, and 11.

Answer 43. Exchanging 8 with 12 would place 12 above 20, leaving a max-heap violation. Exchanging with 20 places 20 above both 12 and 8. Only the lower position receiving 8 may need further repair.

Answer 44. Most nodes are close to the bottom: leaves need no repair, their parents move at most one level, and progressively fewer nodes can move farther. Summing actual possible distances gives a constant times n , so bottom-up construction is $\Theta(n)$. Repeated insertion can make many new values climb $\Theta(\log n)$ levels, giving $\Theta(n \log n)$ worst-case total time.

13.7 Solutions 45–48: heap sort

Answer 45. Build from $[3, 1, 4, 2]$. Repair position 2 to obtain $[3, 2, 4, 1]$, then position 1 to obtain $[4, 2, 3, 1]$. Now extract and repair as follows:

Round	After root/end exchange	After active-heap repair
1	$[1, 2, 3 \mid 4]$	$[3, 2, 1 \mid 4]$
2	$[1, 2 \mid 3, 4]$	$[2, 1 \mid 3, 4]$
3	$[1 \mid 2, 3, 4]$	$[1 \mid 2, 3, 4]$

Final result: $[1, 2, 3, 4]$.

Answer 46. Ascending in-place heap sort uses a max-heap, because it puts each extracted maximum at the right end. Ascending output created by repeatedly removing and appending uses a min-heap, because the output list receives the smallest remaining value first. The location receiving each removed value changes which extreme is useful.

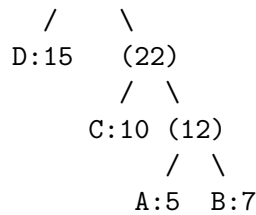
Answer 47. The suffix already contains values in their final positions. Including it in heap repair could move those values back into the unfinished region. After each complete round, the active region is a max-heap; the suffix is sorted in ascending order and every suffix value is at least as large as every active value.

Answer 48. Worst-case time is $\Theta(n \log n)$: linear construction followed by a linear number of removals that can each require logarithmic repair. Auxiliary space is $O(1)$ with iterative repair in the same array. Ordinary heap sort is not stable because long-distance exchanges can reverse the relative order of equal-key records, as the $2_a, 2_b$ example in the notes demonstrates.

13.8 Solutions 49–54: Huffman coding

Answer 49. Merge $5 + 7 = 12$, then $10 + 12 = 22$, then $15 + 22 = 37$. One valid choice puts D on the root's left, C on the next left, and A and B at the deepest pair of leaves:

(37)



Assign 0 left and 1 right. Codes are D=0, C=10, A=110, B=111. Total payload cost is

$$15(1) + 10(2) + 5(3) + 7(3) = 71 \text{ bits.}$$

Equivalently, sum merge weights: $12 + 22 + 37 = 71$. Total frequency is 37, giving average length $71/37 \approx 1.919$ bits per symbol.

Answer 50. Four symbols require 2 bits each with fixed-length coding. For 37 occurrences, that gives $2(37) = 74$ bits. Huffman saves $74 - 71 = 3$ payload bits, or $3/74 \times 100\% \approx 4.05\%$. Additional codebook storage is excluded from this calculation.

Answer 51. Yes, it is prefix-free: no listed code is the start of another. BAD encodes as $10 \mid 0 \mid 111 = 100111$. The stream 011010111 splits at completed codes as $0 \mid 110 \mid 10 \mid 111$, giving A, C, B, D, or ACBD.

Answer 52. The stream 01 can mean B, whose code is 01, or A followed by C, whose codes are 0 then 1. Since A's code is a prefix of B's code, immediate leaf-style decoding is not possible with this proposed codebook.

Answer 53. Merge two 1s to make 2, merge the other two 1s to make another 2, then merge $2 + 2 = 4$. Every leaf has depth 2, so all codes have length 2. The total payload is $4(2) = 8$ bits, also equal to $2 + 2 + 4$. Codes can be 00, 01, 10, 11 assigned to symbols in different ways; exact strings need not match.

Answer 54. Merge $2 + 3 = 5$, then $5 + 7 = 12$, then $9 + 12 = 21$. Total cost is $5 + 12 + 21 = 38$. The leaves of weights 2 and 3 have depth 3, weight 7 has depth 2, and weight 9 has depth 1. Thus $2(3) + 3(3) + 7(2) + 9(1) = 38$. Each merge adds one level to all symbols in its merged group, contributing exactly that group's weight to the eventual encoded cost.

13.9 Solutions 55–60: analysis and synthesis

Answer 55. Since $1023 = 2^{10} - 1$, the tree is perfect, with height 9 and 10 occupied levels. A longest root-to-node search path inspects 10 nodes. A complete traversal visits all 1023 nodes. A comparison count here means comparing the query with one stored node key, not counting individual machine instructions.

Answer 56. Ordinary BST search can take $O(n)$ in a chain. AVL search takes $O(\log n)$ in the worst case because height is controlled. Searching a heap for an arbitrary value takes $O(n)$ in the worst case because the parent-child priority rule does not generally tell you which one subtree contains the value. A short path helps only when you know which path to follow.

Answer 57. Build-heap costs $\Theta(n)$. The removals cost at most $O(\log n), O(\log(n-1)), \dots$, with a worst-case total of $\Theta(n \log n)$ for the standard process. Therefore the combined worst-case bound is $\Theta(n \log n)$. The repeated removals grow faster than the one-time linear construction.

Answer 58. A min-heap is natural because the earliest event has the smallest time and should be at the root. If ordered range queries are central, a balanced search tree such as an AVL tree

becomes useful because it supports key order and can skip values outside a queried interval. Equal timestamps need an explicit duplicate or tie-breaking policy.

Answer 59. Frequency counting costs $O(N)$ under constant-time counter access. Constructing the tree with a min-heap costs $O(k \log k)$. Combined cost is $O(N + k \log k)$ before writing the encoded payload. N counts every occurrence, while k counts different symbols; a very long message can use a small alphabet.

Answer 60. The corrected claims are:

- (a) A complete traversal is $\Theta(n)$ even when height is logarithmic, because all nodes are visited.
- (b) Every AVL tree is a BST, but an ordinary BST need not be balanced.
- (c) A max-heap array obeys parent-child priority, not descending order between all consecutive entries.
- (d) Preorder and postorder do not uniquely determine a general binary tree; single-child side information can be missing.
- (e) Big O is an upper growth bound. The case being bounded must be stated separately.

13.10 Additional implementation checks, with explanations

These are short reading questions about the C already shown, not requests to write long programs. Try answering each question before reading its explanation.

Check A: why save the returned root? Suppose an empty tree calls `bst_insert(root, 50)`; but discards the returned reference. Has the caller's root changed?

Explanation. No. The function receives a copy of the caller's reference and returns the newly created node. The caller must assign that result to its own root. The same principle matters when deletion removes the root or an AVL rotation promotes a child.

Check B: when does a traversal actually visit? If the print in the inorder snippet moves above both child calls, which order does it represent?

Explanation. Preorder. The name of the function does not determine the order; the placement of the visit action does. If making that change into a reusable preorder function, its recursive child calls must apply the same preorder rule too.

Check C: what should survive deletion? A node containing 60 has one right child, 65. Which reference must be saved before freeing 60?

Explanation. Save the reference to 65. Return that reference for the parent to attach. After 60 is freed, reading 60's child fields is invalid because that node's storage has been released.

Check D: why update the lower node first? After rotating right at z , why refresh z 's height before y 's?

Explanation. z becomes a child of y . The new height of y depends on z 's new height. Updating y first would use an outdated child height.

Check E: what does the heap-size argument count? After heap sort places the largest value at index 4, the active indices are 0, 1, 2, 3. What count should sift down receive?

Explanation. Four, not five and not three. The argument is the number of active items, not the old total and not the last active index. This matches `sift_down(a, end, 0)` when `end` is 4.

Check F: does ordinary BST insertion preserve AVL balance? Can the ordinary `bst_insert` function replace `avl_insert` merely because both take a `Node *`?

Explanation. No. Sharing a record layout does not mean sharing the balancing rules. Ordinary insertion finds a valid ordered position. AVL insertion additionally updates heights and performs the required rotations.

Final self-test checklist

You are ready to revise from the compact sheets when you can do all of the following without looking up a procedure:

- Explain every part of a small tree using parent, child, leaf, depth, height, and subtree.
- Distinguish full, complete, perfect, BST, AVL, and heap rules with a drawing.
- Convert a complete binary tree between its picture and its array positions.
- Produce all four traversals and reconstruct a tree from a valid inorder pair.
- Search, insert, and delete in a BST, including a two-child deletion with a surviving replacement child.
- Identify and perform LL, RR, LR, and RL repairs while keeping every middle subtree.
- Explain the difference between sift down, sift up, and bottom-up heap construction.
- Trace heap sort while keeping the active heap boundary correct.
- Build a Huffman tree, assign a valid codebook, decode a bit stream, and calculate its cost in two ways.
- Explain each time and space bound using what the operation actually does.
- Write each main procedure as numbered Start-to-End steps, including the empty-input case and the condition for repeating.
- Read a short C function and connect its decisions, subtree returns, and changed references to the corresponding tree diagram.